Building an LLM inference engine

Companion study notes for Parts 0-9, with a CUDA primer

Samir Sarwar

Read alongside the videos and repository. Early chapters describe their video milestones; Parts 8 and 9 also explain the current paged-cache engine. The implementation uses fixed Llama 3.2 1B dimensions and a 2,048-token context. These notes are explanations, not GPU correctness or performance results.

Contents

AI Inference, Simply Explained - Part 0 Notes **- p. 2**

Building an Inference Engine - Notes **- p. 5**

Embedding Gather - Video Notes **- p. 9**

RMSNorm Kernel - Part 3 Video Notes **- p. 17**

Rotary Position Embedding (RoPE) - Part 4 Video Notes **- p. 21**

Q, K, V Projection with cuBLAS - Part 5 Video Notes **- p. 26**

Part 6: Finishing the Self-Attention Sublayer **- p. 30**

Part 7: Feed-Forward Network, SwiGLU, and the First Predicted Token **- p. 36**

Part 8: Prefill, Decode, and the KV Cache **- p. 44**

Part 9: Static and Continuous Batching **- p. 54**

CUDA Kernels - Getting Started Notes **- p. 66**

Copyright (c) 2026 Samir Sarwar. All rights reserved. Personal reading and study are permitted. See LICENSE-NOTES in the repository for the full terms. Source-code excerpts retain their source license.

AI Inference, Simply Explained - Part 0 Notes

Video: AI inference as simply as possible. These notes set up the ideas used by the later code videos. The working version of my thoughts lives in `notes.md`; this is the version I would want beside me while following the series.

What am I doing?

I'm building a small LLM inference engine with C++ and CUDA to explain what happens between typing a prompt and receiving the next token. This project targets one particular model, Llama 3.2 1B Instruct; it is not a general vLLM replacement.

Here is the whole trip before we zoom in:

```
text prompt
  -> tokenizer -> token IDs
  -> embedding lookup -> hidden vectors
  -> 16 Transformer layers -> final hidden vector
  -> output scores for vocabulary tokens
  -> choose next token ID -> tokenizer decode -> text
```

These notes introduce the whole model before the implementation videos build it piece by piece. The later Part 6 snapshot (`ec066a7`) stopped after layer 0's attention residual; Part 7 completed all 16 layers and the first-token prediction. The current `src/main.cpp` also generates responses with a KV cache, batching, and paged decode attention. Read each companion note in the context of its video's milestone.

Wtf is inference?

The simplest answer I found is: **inference is using a trained model on new input to produce a prediction**. For a causal LLM, that prediction is usually a score for every possible next token. We pick a token from those scores, append it to the sequence, and run the model again to generate more text.

During **training**, an algorithm repeatedly adjusts the model's weights based on examples and an objective. During **inference**, we keep those learned weights fixed and use them in the model's operations. Loading a model file is therefore only the start of inference; the file does not execute itself.

I initially thought of a model as a file full of floating point numbers, which is a useful physical picture. The more complete picture is:

- **Architecture**: the blueprint saying which operations run, in which order, and with which shapes.
- **Weights**: the learned numbers those operations use. Our `model.safetensors` stores them.
- **Activations**: the temporary values produced for *this prompt* as it flows through the model. Unlike weights, these change from request to request.
- **Inference engine**: the program that loads the weights, implements the architecture, manages memory, and executes the operations.

One clarification from the working notes: the model predicts a **token**, not necessarily a whole word. A token can be a word, part of one, punctuation, or another text/byte piece. Also, an engine can affect the numerical result if it changes precision, operation order, or sampling. The main job is to execute the intended computation correctly and efficiently.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

The chef analogy that helped me

Think of a chef learning recipes, and the kitchen being the engine:

- **Training** is culinary school. The chef practices and changes what they know. That learned knowledge is like the weights.
- **Inference** is asking the trained chef to make a meal from new ingredients. The chef uses what they learned to produce a result.
- **The inference engine** is the kitchen: equipment, space, and a process for getting ingredients to the chef and food back out. A better kitchen can serve more meals faster and use resources better.

There is a limit to the analogy. The real program follows exact numerical operations; it does not improvise the model architecture. We still need both the recipe/blueprint **and** the trained weights. A raw SafeTensors file cannot run like an executable, and an empty architecture has nothing learned to apply.

Why an inference engine at all?

Suppose I have the weight file. I still need code to:

1. Read its tensor metadata and put the weights where computation can use them.
2. Turn input text into the token IDs expected by this model.
3. Run operations in the model's exact order, including the right tensor shapes.
4. Keep intermediate values and, eventually, the attention KV cache in memory.
5. Choose a next token and translate its ID back to readable text.

Serving also brings the engineering problem: latency for one prompt, throughput across prompts, and GPU memory use. That is why the "engine" part matters even when the mathematical model is already trained.

Why C++ and CUDA?

Well, we wanna be fast and see how the hardware gets used. C++ handles the host-side control flow, file loading, tokenizer, allocations, and kernel launches. CUDA lets us express numerical work for an NVIDIA GPU. A lot of LLM work consists of matrix multiplications and other operations over huge arrays, which give a GPU plenty of parallel work.

This is not because every GPU task is faster than every CPU task. Moving data between CPU memory and GPU memory costs time. That is why we load large, mostly fixed model weights to VRAM once and keep intermediate activations there as operations run. We send small input token-ID arrays to the GPU when needed.

```
CPU / host RAM: open files, parse metadata, tokenize, schedule work
          | copy input and weights
          v
GPU / device VRAM: tensors and CUDA kernels doing model math
```

The CUDA getting-started notes unpack blocks, threads, shared memory, and launches before the kernel videos.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

What the model is doing, in rough order

For the Llama 3.2 1B Instruct shape used in this project:

```
token IDs [T]
  -> embeddings [T, 2048]
  -> 16 decoder layers, each using attention and an MLP
  -> final RMSNorm [T, 2048]
  -> vocabulary scores [T, 128256]
  -> select a next-token ID
```

`T` is the number of prompt tokens. `128256` is the vocabulary size, and `2048` is the hidden-vector width. Each decoder layer updates those hidden vectors. The operations are *not* interchangeable: RMSNorm, Q/K/V projections, RoPE, masked attention, residual additions, and the MLP must run in the model's intended order. Part 1 has the architecture and weight names; Part 2 shows the first actual GPU operation.

There is a subtle point about the last step. The model produces **logits**, which are scores, not already-selected words. Greedy decoding takes the largest logit with `argmax`; other sampling strategies may choose differently. The shapes above show what the model could calculate at every prompt position. Our generation code applies the final norm and LM head only to the last prompt row, because that row predicts the first response token.

Prompt processing and generating the next token

Two phases will matter later:

- **Prefill**: run the whole prompt through the model, with causal attention, to establish hidden states and cache the K/V values for its positions.
- **Decode**: append one new token and run enough work to predict the next one, reusing the prior K/V cache rather than recomputing the whole prompt each time.

Why a **causal** mask? When predicting the next token, a position should only use tokens at that position or earlier. It must not peek into the future text. Part 8 explains how the completed prefill path retains each layer's K/V and how decode reuses them.

The main distinction to remember

```
training:  change the weights
inference: keep the trained weights fixed and calculate predictions
engine:    implement and run that calculation on hardware
```

That's the foundation for the rest of the series. We start with a file of learned numbers, then gradually build the executable path that gives those numbers a useful job.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

Building an Inference Engine - Notes

Companion to Part 1: Loading model weights. This is the loading milestone: later videos add the forward pass, generation, batching, and paging. The current loader still targets this fixed Llama 3.2 1B layout; it does not infer a generic architecture from the checkpoint.

SafeTensors

There are different formats you can download models in, SafeTensors being one of the most popular.

A safetensor file has 3 sections: **header size**, **JSON header**, **tensor data**. Quick aside: a tensor is a multi-dimensional array of numbers, and the tensor data here is the model weights.

- **Header size** is always 8 bytes, an unsigned 64-bit integer stored in little-endian order.
- **JSON header** is just key-value pairs, where the key is a unique tensor name. Every value is itself a JSON with 3 keys:
  - `dtype` - what data type the tensor is stored in
  - `shape` - the dimensions of the tensor
  - `data_offsets` - a start and exclusive end telling us where that tensor's raw numbers live in the data section

What we implement

```
LlamaForCausalLM(
  (model): LlamaModel(
    (embed_tokens): Embedding(128256, 2048)
    (layers): ModuleList(
      (0-15): 16 x LlamaDecoderLayer(
        (self_attn): LlamaAttention(
          (q_proj): Linear(in_features=2048, out_features=2048, bias=False)
          (k_proj): Linear(in_features=2048, out_features=512, bias=False)
          (v_proj): Linear(in_features=2048, out_features=512, bias=False)
          (o_proj): Linear(in_features=2048, out_features=2048, bias=False)
        )
        (mlp): LlamaMLP(
          (gate_proj): Linear(in_features=2048, out_features=8192, bias=False)
          (up_proj): Linear(in_features=2048, out_features=8192, bias=False)
          (down_proj): Linear(in_features=8192, out_features=2048, bias=False)
          (act_fn): SiLUActivation()
        )
        (input_layernorm): LlamaRMSNorm((2048,), eps=1e-05)
        (post_attention_layernorm): LlamaRMSNorm((2048,), eps=1e-05)
      )
    )
    (norm): LlamaRMSNorm((2048,), eps=1e-05)
    (rotary_emb): LlamaRotaryEmbedding()
  )
  (lm_head): Linear(in_features=2048, out_features=128256, bias=False)
)
```

That's pretty much the entire architecture for Llama 3.2 1B Instruct. What it gives us is the operations we need to run plus the data shapes and types we need to use. What it *doesn't* give us is the order of operations or the dtype - it's just an object with code I don't really understand yet.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

For the dtype we can look the model up on HuggingFace: the weights are **BF16**.

> **BF16** is a 16-bit floating point number with 1 sign bit, 8 exponent bits, and 7 fraction bits. FP32 also has 8 exponent bits, so both cover a similar magnitude range, up to about `3.4 × 10^38`; this is not the range `-256` to `255`. FP32 has 23 fraction bits, so BF16 represents fewer distinct values within that range and rounds more coarsely. Storing weights in BF16 halves their memory use compared with FP32. The engine often converts values to FP32 for reductions and other arithmetic, then rounds the stored output back to BF16.

Order of operations

We need the order of operations to produce correct output, since the model is just specific computations in a specific order. For that, search up the architecture - diagrams are good, Sebastian Raschka has a bunch. Let's figure out how I'm supposed to dissect the diagram.

Input text → tokens. A **tokenizer** converts text into pieces the model recognizes. It splits the input into smaller pieces (whole words, pieces of words, punctuation) and assigns each an ID. It can do this because it built a fixed vocab of token pieces during training, each mapping to an ID - the ID value itself is arbitrary. What if it wasn't trained on some input? Then it breaks it down as far as it needs to until the pieces are in its vocab, e.g. `["hyperglob"] -> ["hyper", "gl", "ob"]`.

Tokens → embeddings. Then we hit up a mega table of model weights, the **embedding table**, where the token ID is a row index. `(embed_tokens): Embedding(128256, 2048)` tells us there are 128,256 possible token IDs and 2048 learned weights per token, which form an **embedding vector**.

The pink box is a **transformer block** (also called a layer). There are 16 of them, with a shit ton of operations I don't know yet but will get a clear understanding of as I build more of the engine:

1. RMS Norm
2. Residual connection
3. Masked grouped-query attention:
   - Q projection
   - K projection
   - V projection
   - RoPE with Q projection
   - RoPE with K projection
   - Attention / attention scores
   - Causal mask
   - Softmax
   - Attention scores with V projection
   - O projection (output projection)
4. Residual connection add
5. RMS Norm
6. Feed forward (like in first neural networks, multilayer perceptron):
   - Gate projection (first linear layer)
   - Up projection (second linear layer)
   - SiLU activation - similar to ReLU but looks more like a sigmoid

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

- Down projection (third linear layer)

7. Residual connection add

Then a final RMS Norm, a Linear output, then Argmax. I truly don't got that much of a clue on these computations yet - ngl still just copying from the diagram and docs. Will learn as I go.

An aside on memory

Data lives on **host** or **device**.

- **Host** = the PC/CPU. Big slow memory (DRAM), separate from the CPU, plus the CPU's own tiny fast on-chip SRAM (L1/L2 cache) for quick access to recently used memory.
- **Device** = the GPU. Also has big slow memory (VRAM) and small fast SRAM.

We want the large computations on the GPU, which means it needs access to the weights, input tokens, and intermediate results - but the GPU can't directly touch host DRAM. So we copy: **CPU DRAM → GPU VRAM**.

At the low level that's not as easy as it sounds. On the CPU we create a variable and write to it, compute how many elements it holds, multiply by the size of the type to get total bytes. Then on the GPU we allocate memory so we have somewhere to put it, and copy it over. Now the data is in device VRAM and usable in GPU computations. CUDA has its own functions for this - `cudaMalloc` and `cudaMemcpy` - which are pretty similar to ones I worked with in C.

Loading the model

First step of inference is actually loading the model, i.e. copying weights from CPU to GPU memory like we talked about above. The model is a single `model.safetensors` file: header size, JSON header, tensor data (many tensors).

Opening the file. In C we'd use `fopen` or `open()`; in C++ we use `std::ifstream` - input file stream, an object that lets the program read data from the file. We pass `ios::binary` to say this is raw binary, so every byte is preserved exactly. Without it the compiler sometimes translates special chars or line endings, which corrupts bytes and makes the offsets wrong.

Why we read the header. The JSON header is metadata on every tensor in the model. There are 16 transformer layers, each with 9 tensors, and each tensor occupies some space in the file recorded by its offsets. We need this because at some point we have to use these weight values and we can't keep reading from the file - super slow. It has to go to VRAM. Reading (and printing) the header also shows us the naming convention for tensors, which lets us quickly find what we're looking for later.

Reading the header. The file starts with the header size as a `uint64_t` (8 bytes) telling us how big the JSON header is. We call `.read(destination, bytes_to_read)` on the opened file for those first 8 bytes. Now we know the JSON header size, so we call `.read` again for that many bytes - the file pointer is already sitting at the start of the JSON header (8 bytes in), so we don't have to handle that. `.read` needs a `char` destination buffer, i.e. a string, so we make a string of size `headersize` initialized to `\0` and read into it.

Parsing the header. We use the JSON library we imported to turn the header string into a JSON object with `json::parse`. We need the offsets for 2 things:

1. The **max offset** tells us how much space the tensor data takes up - we know the header size but not the size of the raw weights.
2. The **start offsets** are needed later to build pointers, since we'll allocate all this on the GPU and want pointers on the CPU so we can quickly access GPU memory.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

To store them we use a hashmap (`unordered_map` in C++) keyed by `string` with `uint64_t` values. We initialize `max_offset` to zero since we check it every iteration.

Then we use a structured-binding for loop over the name and `tensor_info` of each header item. This is where having printed the header pays off - you know exactly what you're looking for. The offsets live at `"data_offsets"`, which is an array of two values (start and end). Grab them with `data_offsets.at(...)`, convert to `uint64_t`, then `emplace` them into the hashmap and recompute `max_offset` by checking this tensor's end offset against the current max.

Reading the tensor data. Relatively simple thanks to `max_offset`: the file pointer is already at the start of the raw tensor data and we know how large it is, so just call `.read` again. Where do we store it? A `vector<char>` - that's what `.read` expects, and it matches the file's exact raw byte layout. The actual values are BF16, which is just two contiguous 8-bit chars.

Getting it onto the GPU. Allocate first with `cudaMalloc`, which reserves a block on the GPU and sets a pointer to its start - so before that we initialize a `void` pointer to null so it can be assigned. Then `cudaMemcpy` runs the actual copy. Pretty simple params since we've already made everything it needs.

Hashmap vs struct. Initially I made a hashmap of pointers, each pointing to where that weight lives in GPU memory. That's fine for the initial load, but during inference we go through all 16 layers, and this approach means string construction, hashing, and a hashmap lookup every time - slow. Instead, a **struct** holding all the potential values for a layer, indexed by layer number, gives a much quicker lookup and no string construction in the loop.

That just means handling the construction on initial load instead. So: we create BF16 pointers to specific values, and for the tensors that repeat in every transformer layer we make an array of 16 BF16 pointers, one per layer's version of that tensor. Then we loop through each layer, do the math to figure out where each pointer should point, and save it in the struct so it can be called from CPU memory later.

First we get the **base** - where the memory points to on the GPU, which `cudaMalloc` returned to us and which we store in the weights struct. Then we compute each tensor's location from the base plus the offset hashmap, for every tensor in every layer.

By the way: we need the weights outside this function, so we pass the struct by reference into the loading function.

The struct approach works because we're building this specifically for Llama 3.2 1B - we know the architecture, and we printed the JSON header to see how the tensors are named.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

Embedding Gather - Video Notes

Companion to Part 2: Embedding gather. The host allocation examples describe this video's milestone. The current engine keeps shared scratch buffers in `InferenceBuffers` across prefill and decode calls; the embedding kernel's lookup and thread mapping are the same.

What this video is covering

This is the first actual model operation in the inference engine. We already did the setup work:

1. Load the Llama weights from `model.safetensors`.
2. Copy those weights into GPU memory.
3. Build CPU-side pointers that let us find a specific weight tensor quickly.
4. Tokenize a prompt into a list of token IDs.

Now we need to turn those token IDs into values the Transformer can actually do math with. That is the embedding gather.

The flow so far is:

```
input text
-> tokenizer
-> token IDs
-> copy token IDs to GPU
-> embedding gather
-> activation vectors
-> Transformer layers
```

The important distinction is that a token ID is not a meaningful number to the neural network. Token ID `9906` is not numerically similar to token ID `9907` just because the integers are close. The IDs are labels assigned by the tokenizer. We need to use each ID as an address into the model's learned embedding table.

The embedding table

Our Llama 3.2 1B model describes this tensor as:

```
(embed_tokens): Embedding(128256, 2048)
```

That gives us the shape:

```
[128256 rows, 2048 columns]
```

- There are `128256` possible token IDs in the vocabulary.
- Every token ID gets one row.
- A row has `2048` BF16 values.
- That row is called the token's embedding vector.

Think of this table as a learned dictionary. The tokenizer says, "this piece of input is token 9906." The embedding table says, "the model's learned starting representation for token 9906 is this exact 2048-number vector."

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

The values were learned in training. We do not calculate them during inference. Our job is just to find the right row quickly and move it into the activation buffer.

For a prompt with `T` tokens, the shapes are:

```
token IDs:       [T]
embedding table: [128256, 2048]
activations:     [T, 2048]
```

Example with a small fake table:

```
token IDs = [2, 0]

embedding table
row 0: [0.1, 0.2, 0.3, 0.4]
row 1: [0.5, 0.6, 0.7, 0.8]
row 2: [0.9, 1.0, 1.1, 1.2]

activations after gathering
row 0: [0.9, 1.0, 1.1, 1.2]
row 1: [0.1, 0.2, 0.3, 0.4]
```

We are not multiplying anything here. We are selecting rows. This is why it is called an embedding lookup or an embedding gather.

Where the embedding table comes from

The model loader already copied all raw tensors from the SafeTensors file into one large GPU allocation. Then it creates direct pointers to the individual tensors we need.

```
weights.embed_tokens = reinterpret_cast<__nv_bfloat16 *>(
    base + offsets.at("model.embed_tokens.weight"));
```

`base` is the start of the whole model-weight allocation on the GPU. The SafeTensors header gave us the byte offset of `model.embed_tokens.weight`. Adding the offset to the base gives us a pointer directly to the first BF16 value in the embedding table.

This matters because the table stays on the GPU after startup. We should not copy 128,256 by 2,048 embedding weights from CPU to GPU for every prompt. That would be a ridiculous amount of repeated transfer. We load weights once, then reuse them for every inference request.

Prefill setup on the host

`prefill` is the host-side function that gets a full prompt ready for model computation.

1. Allocate space for token IDs on the GPU

```
int *token_id_gpu = nullptr;

cudaMalloc(&token_id_gpu, token_ids.size() * sizeof(int));
```

`token_ids` is a CPU `std::vector<int>`. `cudaMalloc` reserves enough VRAM for the same number of `int` values and puts the starting GPU address into `token_id_gpu`.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

2. Copy token IDs from host to device

```
cudaMemcpy(
    token_id_gpu,
    token_ids.data(),
    token_ids.size() * sizeof(int),
    cudaMemcpyHostToDevice);
```

The GPU needs the IDs because the kernel will read them. `token_ids.data()` is the CPU source, `token_id_gpu` is the GPU destination, and `cudaMemcpyHostToDevice` makes the direction explicit.

The token IDs are tiny compared with the weights. Copying a short prompt's IDs is cheap. Copying the massive embedding table every time would not be.

3. Allocate output activations

```
__nv_bfloat16 *activations_gpu;

cudaMalloc(
    &activations_gpu,
    token_ids.size() * 2048 * sizeof(__nv_bfloat16));
```

For every token we need 2,048 BF16 output values. This allocation is where embedding gather will write its result.

The output is called an activation buffer because it contains the model's current working values. The embedding table contains fixed learned weights. Activations are produced from the current prompt and change for every prompt.

4. Launch the gather

```
launchEmbeddingGather(
    token_id_gpu,
    weights.embed_tokens,
    activations_gpu,
    token_count);
```

At this point every input needed by the kernel is in VRAM:

- `token_id_gpu` contains the input IDs.
- `weights.embed_tokens` points to the fixed model weights.
- `activations_gpu` is writable output storage.
- `token_count` describes the input shape.

The next operation in the current engine is RMSNorm. It reads these activation vectors and normalizes each token's 2,048 values before the first attention layer.

What a CUDA kernel is

A regular C++ function runs on the CPU. In simple terms, it runs one instruction stream at a time, although modern CPUs can still do a few things in parallel.

A CUDA kernel is a function we launch on the GPU. The GPU runs many copies of that same function at the same time, using many lightweight threads. Each thread needs a small independent piece of work.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

For embedding gather, the output contains:

```
token_count * 2048 values
```

The simplest mapping is one thread per output value.

```
thread 0    writes output value 0
thread 1    writes output value 1
thread 2    writes output value 2
...
thread N    writes output value N
```

Each thread only needs to read one token ID, read one BF16 weight, and write one BF16 activation. No thread needs another thread's result. That is an ideal first CUDA kernel because the work is embarrassingly parallel, meaning it is so independent that there is no coordination problem yet.

The embedding gather kernel

```
__global__ void embeddingGatherKernel(const __nv_bfloat16 *embed_tokens,
                                      __nv_bfloat16 *activations_gpu,
                                      const int *token_id_gpu,
                                      int token_count)
```

`__global__` says this is a CUDA kernel. It is called by host code, but it executes on the GPU.

The arguments are pointers to GPU memory, not regular CPU arrays:

- `embed_tokens` is the read-only embedding table.
- `activations_gpu` is the output activation buffer.
- `token_id_gpu` is the read-only list of input token IDs.
- `token_count` is a normal integer passed by value.

`const` on the two input pointers is useful because it tells us, and the compiler, that this kernel should not modify those inputs.

Give each thread one output index

```
const int index = blockIdx.x * blockDim.x + threadIdx.x;
```

CUDA gives every thread three built-in variables here:

- `threadIdx.x`: this thread's location inside its block.
- `blockIdx.x`: this block's location inside the grid.
- `blockDim.x`: the number of threads in one block.

The calculation flattens the block and thread positions into one unique index. For blocks with 256 threads:

```
block 0: thread 0 to 255   -> global index 0 to 255
block 1: thread 0 to 255   -> global index 256 to 511
block 2: thread 0 to 255   -> global index 512 to 767
```

This index maps directly to the flattened output buffer.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

Count the total output values

```
const int total_values = token_count * 2048;
```

The logical activation shape is `[token_count, 2048]`. GPU memory is a flat sequence of values, so the total number of values is just rows times columns.

`2048` is the hidden size for this exact model. It is currently intentionally hard-coded. Once the engine supports model configs more generally, this should come from the model configuration rather than appearing in multiple places.

Protect against extra threads

```
if (index >= total_values)
{
    return;
}
```

We launch whole blocks of threads, but `total_values` will not always divide evenly by the block size. The launcher rounds up so that there are enough threads. That means the final block can have threads with no valid output value.

This check makes those threads exit. Without it, they could read or write beyond the allocated memory, which is a bug even if it looks like it works sometimes.

Convert the flat index back into row and column

```
int token_position = index / 2048;
const int hidden_index = index % 2048;
```

The output is logically a 2D matrix but physically a flat array. Division finds the row, and modulo finds the column.

For example, if `index` is `4099`:

```
token_position = 4099 / 2048 = 2
hidden_index   = 4099 % 2048 = 3
```

So this thread owns the fourth hidden value for the third token in the prompt.

Find the vocabulary ID for this prompt position

```
const int token_id = token_id_gpu[token_position];
```

Do not mix up `token_position` and `token_id`.

- `token_position` is where the token appears in the prompt: first, second, third, etc.
- `token_id` identifies which row in the vocabulary table to use.

For example, the third prompt position might contain token ID `9906`. We use position `2` to read the input ID, then we use `9906` to select a row in the embedding table.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

Gather one learned value

```
activations_gpu[index] = embed_tokens[token_id * 2048 + hidden_index];
```

This line is the actual embedding gather.

`token_id * 2048` moves to the start of the selected row. Adding `hidden_index` selects one value inside that row. The thread writes that BF16 value into the matching location in the activation buffer.

In matrix notation, the line means:

```
activations[token_position, hidden_index]
    = embedding_table[token_id, hidden_index]
```

The flattened index is just the low-level way to write the same idea in contiguous memory.

Why the launcher exists

The kernel is the GPU work. The launcher is regular host-side code that decides how much GPU work to start and then starts it.

```
cudaError_t launchEmbeddingGather(
    const int *token_id_gpu,
    const __nv_bfloat16 *embed_tokens,
    __nv_bfloat16 *activations_gpu,
    int token_count)
```

Keeping this as a separate function is useful for a few reasons:

1. `main.cpp` can say "run embedding gather" without knowing the CUDA launch details.
2. The launch geometry lives close to the kernel it belongs to.
3. The rest of the engine gets a normal C++ function that returns a `cudaError_t`.
4. Later we can tune block size or use CUDA streams without rewriting the prefill logic.

The launcher chooses 256 threads per block:

```
const int threads_per_block = 256;
```

There is no universal magical block size. 256 is a normal starting choice that is large enough to expose lots of parallel work but still fits the hardware's scheduling model well. Performance tuning is later. Correctness and a clear work mapping come first.

Then it calculates how many blocks are needed:

```
const int blocks =
    (total_values + threads_per_block - 1) / threads_per_block;
```

This is ceiling division using integer math. It makes sure we never launch too few threads.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

For example:

```
total values = 2,050
threads/block = 256
blocks = (2050 + 256 - 1) / 256 = 9
threads launched = 9 * 256 = 2,304
```

There are 254 extra threads. That is why the bounds check in the kernel is necessary.

Finally we launch the kernel:

```
embeddingGatherKernel<<<blocks, threads_per_block>>>(
    embed_tokens,
    activations_gpu,
    token_id_gpu,
    token_count);
```

The triple angle brackets are CUDA launch syntax:

```
kernel<<<number of blocks, threads per block>>>(arguments)
```

This line does not mean the CPU waits for all GPU work to finish. CUDA kernel launches are normally asynchronous. `cudaGetLastError()` checks whether the launch itself was accepted. A later `cudaDeviceSynchronize()` is where the CPU waits and can surface an execution error from the GPU.

Why this kernel maps well to the GPU

This is mostly a memory movement operation. For each output value, we read one BF16 value and write one BF16 value. There is very little math.

For one embedding row, adjacent threads access adjacent hidden values:

```
thread 0 -> row[token ID][0]
thread 1 -> row[token ID][1]
thread 2 -> row[token ID][2]
...
```

Since the block size of 256 divides the hidden size 2048, each block stays within one 2,048-value embedding row. This is a clean contiguous access pattern. GPUs prefer nearby threads accessing nearby memory because hardware can combine those requests efficiently. This is called coalesced memory access.

The more complicated operations later will need threads to share partial results and synchronize. This gather kernel does not. Every output position is independent, so it is a good first step before RMSNorm, reductions, matrix multiplication, and attention.

What embedding gather does not do

Embedding gather gives every token its starting learned vector. It does not understand the rest of the prompt yet.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

If the same token appears twice, its raw embedding row is the same both times. Context gets added later when the transformer layers use attention to let every token combine information from earlier tokens.

The embedding table gives a starting point. The transformer turns those starting vectors into context-aware hidden states.

End state of this video

At the end, the important mental model should be:

```
Tokenizer creates IDs.
IDs are addresses, not model input values by themselves.
Embedding gather uses every ID to select one learned 2048-value row.
The gathered rows become activations on the GPU.
Those activations are the input to the first Transformer operation.
```

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

RMSNorm Kernel - Part 3 Video Notes

Video: Writing the RMSNorm kernel. This picks up after embedding gather. The code to read alongside it is `rmsNormKernel` and `launchRmsNorm` in `src/kernels.cu`, called from `prefill` in `src/main.cpp`.

The host snippets describe the Part 3 milestone, when prefill stopped after the first norm. The current engine reuses this kernel throughout all 16 layers, the final norm, and decode, with persistent scratch buffers.

What this video is covering

We have token IDs, and embedding gather has turned each ID into a 2,048-value hidden vector. The next operation in the first Transformer layer is RMSNorm. It runs **independently for each token's hidden vector**. If the prompt has `T` tokens, our input and output are both `[T, 2048]`.

```
token IDs [T]
  -> embedding gather -> activations [T, 2048]
  -> RMSNorm with layer 0's input_layernorm weights
  -> normalized activations [T, 2048]
  -> later: Q/K/V projections
```

The word "hidden" just means these numbers are the model's internal representation, rather than text we can read directly. `activations_gpu` contains them already. RMSNorm does **not** look up token IDs or copy the embedding table again. That setup belonged to the previous step.

What are we actually normalizing?

In my working notes I described this as getting the average of the vector close to 1. More precisely, RMSNorm makes the **root mean square magnitude** of the vector close to 1 *before* applying a learned scale. It does not make every value close to 1, and it does not subtract the ordinary mean.

For one hidden vector `x` of width `D = 2048`, with learned weight vector `g` and epsilon `eps = 1e-5`:

```
mean_square = (x[0]^2 + x[1]^2 + ... + x[D-1]^2) / D
inverse_rms = 1 / sqrt(mean_square + eps)
y[j] = x[j] * inverse_rms * g[j]
```

The small `eps` prevents division by zero and keeps very small inputs numerically safe. `g[j]` is one learned weight for hidden column `j`, shared across token positions for this particular normalization layer. Each Transformer layer has **its own** `input_layernorm.weight`, and another weight vector for the norm after attention. There is also a final norm at the end of the full model.

Small example from my notes

Let `x = [1, 2, 3, 4, 5]` and, just for the illustration, all learned weights be `1`. The squared values are `[1, 4, 9, 16, 25]`, their mean is `55 / 5 = 11`, and `sqrt(11)` is about `3.3166`. Ignoring the tiny epsilon, the normalized vector is:

```
[1, 2, 3, 4, 5] / 3.3166
    ≈ [0.302, 0.603, 0.905, 1.206, 1.508]
```

Some elements are still greater than 1. What became 1 is the vector's RMS magnitude. With a nontrivial learned `g`, the output changes column by column. That last multiply is part of RMSNorm, not an optional detail.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

My "brother - man + woman ≈ sister" analogy was meant to give an intuition for a vector's direction, but it is not a rule that this model's actual token embeddings obey. Also, dividing by one scalar preserves direction, while multiplying by a different learned `g[j]` in every dimension can change it. It is better to understand RMSNorm directly from the formula.

Why use RMSNorm here?

In a deep model, activations pass through many layers. Normalizing the size of each token's hidden vector gives the following projections a more controlled input scale. It is one part of how the trained architecture behaves, so we implement it because Llama expects it, not because we can arbitrarily insert it anywhere.

LayerNorm is a related operation: it subtracts the vector's mean and divides by a standard-deviation-like quantity before applying learned parameters. RMSNorm omits the centering step. I said in the rough notes that centering "didn't help"; that is too absolute. The useful distinction for *our code* is simply that RMSNorm needs a **sum of squares**, not a sum of values and a mean subtraction.

Where its inputs come from

In `prefill`, the host has already allocated `activations_gpu` and launched embedding gather. It then allocates a second `[T, 2048]` BF16 buffer, `normalized_gpu`, for the output:

```
const size_t activation_bytes =
    static_cast<size_t>(token_count) * 2048 * sizeof(__nv_bfloat16);

cudaMalloc(&normalized_gpu, activation_bytes);

launchRmsNorm(
    activations_gpu,
    normalized_gpu,
    weights.input_layernorm[0],
    token_count);
```

`weights.input_layernorm[0]` points into the big weight allocation loaded at startup. It is a vector of 2,048 BF16 learned values. We do not copy the weights again for this prompt. The `0` matters: this video works with the **first** layer's input norm. The completed forward pass loops over all 16 layers and selects the corresponding norm vector each time.

The CUDA mapping: one block per token

Here is the interesting parallelism problem. For a single output `y[j]`, we need the sum of **all 2,048 squared input values** for that token. Threads cannot calculate their outputs completely independently as they could in embedding gather. They have to cooperate on that sum.

The launcher uses:

```
rmsNormKernel<<<token_count, 1024>>>(input, output, norm_weights);
```

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

So one CUDA block handles one token. Within the block, 1,024 threads split the 2,048 hidden values. Each thread handles two values, one in each half:

```
const int tid = threadIdx.x;
const int base = blockIdx.x * 2048;

const float x0 = __bfloat162float(input[base + tid]);
const float x1 = __bfloat162float(input[base + tid + 1024]);
```

Example: block 2 handles the third token. Its `base` is `2 * 2048`. Thread 7 reads hidden columns 7 and 1031 of that same token. A different block works on a different token, so it never needs to combine sums with block 2.

The 1,024-thread choice matches this fixed hidden width and the kernel's array size. It is the usual maximum threads per block on the target CUDA hardware, but a device's actual limit should be checked; the code assumes this launch is supported.

Shared memory and the first barrier

Each thread calculates one partial squared sum:

```
__shared__ float partial[1024];

partial[tid] = x0 * x0 + x1 * x1;
__syncthreads();
```

`__shared__` is memory visible to **threads in this block**. All 1,024 entries together represent the sum of squares for all 2,048 values. We convert BF16 inputs to FP32 and keep partial sums in `float` so the reduction loses less precision. The stored model weights and output are still BF16.

`__syncthreads()` is a block-wide barrier: every thread must finish writing its slot before another thread reads a neighbor's slot. I compared it to `pthread_join()` in the rough notes, but a barrier is closer to the truth. The threads keep running after it; we are not ending or joining them.

Tree reduction: adding all the partial sums

If one thread added `partial[0]` through `partial[1023]` on its own, it would leave the other threads idle. Instead, the kernel repeatedly halves the number of useful sums:

```
for (int i = blockDim.x / 2; i > 0; i >>= 1)
{
    if (tid < i)
    {
        partial[tid] += partial[tid + i];
    }
    __syncthreads();
}
```

For the first round, `i = 512`:

```
partial[0]   += partial[512]
partial[1]   += partial[513]
...
partial[511] += partial[1023]
```

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

Then 256 threads combine the remaining 512 sums; then 128, 64, and so on down to 1. With eight starting numbers, the mental model is four pair additions, then two, then one: three reduction rounds rather than a seven-addition serial chain. Our kernel takes ten halving rounds from 1,024 partial sums to one total.

The barrier **inside every round** matters. Imagine thread 0 entering the 256 round and reading `partial[256]` before thread 256 finished its work in the 512 round. We would get an incomplete sum. The barrier prevents that. Once the loop ends, the sum of squares for this token is in `partial[0]`.

One thread finishes the scale; all threads write output

The kernel lets thread 0 calculate the inverse RMS and store it back in shared memory:

```
if (tid == 0)
{
    partial[0] = rsqrtf(partial[0] / 2048.0f + 1.0e-5f);
}
__syncthreads();
```

`rsqrtf(z)` computes `1 / sqrt(z)`. That is the same factor we need to divide each input by its RMS. We synchronize once more so every thread reads the finished factor, then thread `tid` writes its two output columns:

```
const float inverse_rms = partial[0];
const float weight0 = __bfloat162float(norm_weights[tid]);
const float weight1 = __bfloat162float(norm_weights[tid + 1024]);

output[base + tid] = __float2bfloat16(x0 * inverse_rms * weight0);
output[base + tid + 1024] =
    __float2bfloat16(x1 * inverse_rms * weight1);
```

Notice the two meanings of "weight" here. `activations_gpu` holds prompt-dependent input values; `norm_weights` holds fixed learned scale parameters. We normalize **activations**, then multiply by those learned parameters.

Launch errors versus execution errors

`launchRmsNorm` returns `cudaGetLastError()` immediately after the kernel launch. That catches an invalid launch configuration or another launch-time failure. GPU work normally runs asynchronously, so it does not by itself prove every thread completed correctly. `prefill` later calls `cudaDeviceSynchronize()` and checks its result to catch execution failures before freeing buffers.

What this code does today, and what comes next

The kernel is reusable for any token count with this fixed 2,048 hidden width. At this video's milestone, `prefill` called it only for layer 0's first norm; Q/K/V projection was the next step. The later Q/K/V notes show how `normalized_gpu` becomes that projection input. Part 7 completes the forward pass and produces the first next-token prediction.

The thing to keep in mind from this video:

```
one block = one token
one thread = two hidden values
shared memory + barriers = one token-wide sum of squares
inverse RMS + learned per-column scale = normalized BF16 output
```

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

Rotary Position Embedding (RoPE) - Part 4 Video Notes

Video: RoPE explained and a kernel written from scratch. This follows RMSNorm. The implementation to keep open is `initializeRopeTables`, `ropeKernel`, and `launchRope` in `src/kernels.cu`, plus `prefill` in `src/main.cpp`.

The video milestone implements RoPE before connecting Q/K projections. The current source also applies the same kernel during decode, using each sequence's absolute position in the cache.

What problem are we solving?

An embedding lookup gives the same learned starting row to a token ID wherever it appears. The model needs to know something about **where** tokens occur. My rough example was “I bought an Apple watch” versus “Watch an Apple I bought”: the token pieces may overlap, but changing their order changes how they relate. That example is just an intuition, not a promise that the tokenizer gives exactly the same pieces or that the sentences have one clear meaning.

Without positional information, attention based only on content has no built-in way to distinguish a simultaneous rearrangement of tokens. In a causal model, the mask and order of processing already matter, but the attention mechanism still benefits from a direct signal about positions. RoPE supplies that signal to attention by rotating **query and key activations** according to their token positions.

The order in the attention sublayer is:

```
hidden states
  -> RMSNorm
  -> learned Q, K, V projections
  -> rotate Q and K with RoPE; leave V alone
  -> Q·K attention scores, causal mask, softmax
  -> weighted sum of V
```

That last distinction is important. We do not rotate the learned `w_q` and `w_k` **weights** from the SafeTensors file. We rotate the prompt-dependent **Q and K vectors produced by those weights**. At this video's milestone the RoPE table builder and kernel were ready, but `prefill` did not yet create Q/K projection buffers or call `launchRope`. The later Q/K/V notes show that connection.

The two-number picture

Imagine one query has only two coordinates, `(x0, x1)`. At position `p`, RoPE gives this pair an angle `p * ω`, where `ω` is that pair's frequency. The rotation is ordinary 2D geometry:

```
x0' = x0 * cos(p * ω) - x1 * sin(p * ω)
x1' = x0 * sin(p * ω) + x1 * cos(p * ω)
```

We apply the same kind of rotation to a key at its own position. Position `0` has angle `0`, so its pair is unchanged. Positions in this code are **zero-based**: for a `T`-token prompt, the host currently creates `[0, 1, ..., T-1]` with `std::iota`.

My “I walk my dog” picture still helps. If “I” is at position 1 and “dog” is at position 4 in a simplified numbering, the relevant key and query get angles `1ω` and `4ω`. Their comparison depends on the difference `3ω`. If we shift *both* positions forward by two, the angle difference stays `3ω`. We do not explicitly subtract positions in the kernel; the dot product of the rotated vectors produces the relationship.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

Why does the dot product see relative distance?

Let `R(a)` be the 2D rotation matrix for angle `a`. A query `q` at position `m` becomes `R(mω)q`; a key `k` at position `n` becomes `R(nω)k`. Their dot product is:

```
(R(mω)q) · (R(nω)k)
  = qᵀ R(mω)ᵀ R(nω) k
  = qᵀ R((n - m)ω) k
```

The key fact is `R(a)ᵀ = R(-a)`. The comparison therefore depends on the **relative offset** `n - m` for that pair. RoPE uses an absolute position to rotate each vector, but the resulting Q/K score contains relative-position information. It does **not** make word order irrelevant or guarantee that two rearranged sentences mean the same thing.

For causal self-attention, a query at `m` may attend to keys at `n <= m`. The causal mask handles that permission separately; RoPE does not replace it.

The two-number example from my working notes

Let the query be `q = (2, 1)` at position `2`, and the key be `k = (3, 4)` at position `5`. Pick an easy fake frequency `ω = 0.1` radians per token:

```
query angle = 2 * 0.1 = 0.2
key angle   = 5 * 0.1 = 0.5

q' = (2*cos(0.2) - 1*sin(0.2), 2*sin(0.2) + 1*cos(0.2))
   ≈ (1.7615, 1.3774)

k' = (3*cos(0.5) - 4*sin(0.5), 3*sin(0.5) + 4*cos(0.5))
   ≈ (0.7150, 4.9486)

q' · k' ≈ 8.0758
angle difference = (5 - 2) * 0.1 = 0.3
```

That `8.0758` is **one pair's contribution** to a raw Q/K dot product, not a final attention probability. In a full head, the contributions of all pairs are added, the score is divided by `sqrt(head_size)`, the causal mask is applied, and softmax turns allowed scores into attention weights. Also, this particular `query=2, key=5` comparison would be masked in a causal model because the key is in the future. I kept the numbers because they make the rotation clear; swap the positions for an allowed query/key comparison.

A real Llama head has 64 values, not two

Our query head has `64` numbers, so it contains `32` independent 2D rotation pairs. Llama's layout in this kernel is **half split**:

```
head = [q0, q1, ..., q31, q32, q33, ..., q63]
pairs: (q0, q32), (q1, q33), ..., (q31, q63)
```

This is easy to confuse with adjacent pairs `(q0, q1)`, `(q2, q3)`, etc. The kernel explicitly uses `second_index = first_index + head_size / 2`, so use the half-split picture when reading this code. Each pair in the same token uses that token's position, but each pair has a **different frequency**.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

Here's my tiny four-number head example, with fake frequencies chosen just to see what happens:

```
Q at position 3 = [2, 5, 3, 7]
pair 0 = (2, 3), frequency 0.1  -> angle 3 * 0.1  = 0.3
pair 1 = (5, 7), frequency 0.01 -> angle 3 * 0.01 = 0.03

rotate pair 0: (2, 3) -> (1.024, 3.457)
rotate pair 1: (5, 7) -> (4.788, 7.147)

rotated Q ≈ [1.024, 4.788, 3.457, 7.147]
```

Notice how the results go back into their **original half-split columns**. We are not concatenating the pairs as `(1.024, 3.457, 4.788, 7.147)`. In the real model this happens for all 32 pairs of each query and key head.

Where the real frequencies come from

For pair index `i` from `0` to `31`, the unscaled inverse frequency is:

```
ω_i = 1 / (theta ^ (2i / head_size))
theta = 500000
head_size = 64
angle(position, i) = position * ω_i
```

That is what the `inverse_frequencies` loop in `initializeRopeTables` calculates. Pair 0 has `ω_0 = 1`, so it turns relatively fast as position rises. Later pairs have smaller frequencies and turn more slowly. The fake `0.1` and `0.01` above are for learning the math; they are not the two values this Llama configuration uses for its first two pairs.

One full turn is `2π` radians, so a frequency `ω` has wavelength:

```
λ = 2π / ω       (measured in token positions)
```

Large frequency means short wavelength and faster rotation. Small frequency means long wavelength and slower rotation. This is the clean version of the formulas that got mangled in my rough notes.

Llama 3.2's frequency scaling in this code

The table builder applies Llama's long-context scaling to each `ω_i` before calculating sine and cosine. The code uses:

```
original context length = 8192
scale factor = 32
low-frequency factor = 1
high-frequency factor = 4

low-frequency wavelength boundary  = 8192 / 1 = 8192
high-frequency wavelength boundary = 8192 / 4 = 2048
```

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

For each pair's **original** wavelength `λ = 2π / ω`:

Wavelength	Scaled frequency ω'	What happens
`λ < 2048`	`ω`	Fast pairs keep their local detail.
`2048 <= λ <= 8192`	`(1 - s) * (ω / 32) + s * ω`, with `s = (8192 / λ - 1) / 3`	Blend smoothly between the two behaviors.
`λ > 8192`	`ω / 32`	Slow pairs turn even more slowly.

At `λ = 2048`, `s = 1`, so the blend equals the original frequency. At `λ = 8192`, `s = 0`, so it equals `ω / 32`. The inequalities in the C++ branches put the exact boundaries in the middle branch, but the values join smoothly. After scaling, the table angle is `position * ω'`.

This scaling is part of the model configuration, not a property of every RoPE implementation. Also, the current program builds tables for only `MAX_SEQUENCE_LENGTH = 2048` positions. A long-context frequency formula does not by itself make this checkout support arbitrarily long prompts.

Why precompute cosine and sine tables?

The same position and pair frequency are reused across heads and layers. Instead of having every GPU thread compute `cos` and `sin` from scratch, `initializeRopeTables` calculates tables on the **CPU**:

```
cos_table[position, pair] = cos(position * scaled_frequency[pair])
sin_table[position, pair] = sin(position * scaled_frequency[pair])
```

Each table has shape `[max_sequence_length, 32]` in FP32. The function allocates two GPU buffers, copies the CPU tables to them, and returns their GPU pointers through `float **cos_table_gpu` and `float **sin_table_gpu`. The pointers are freed in `main` at the end.

The caller must provide position IDs between `0` and `max_sequence_length - 1`. `launchRope` validates pointer and shape arguments, but it does not check every position ID against the table size. Keeping the prompt within the allocated table is therefore part of using this kernel correctly.

Reading the RoPE kernel from the thread's point of view

The input is a flattened BF16 projection buffer. For each token, its heads are contiguous, and each head has `head_size` values. One thread owns **one pair**, writes its two rotated BF16 results in place, and does not need to synchronize with other threads.

First, the kernel gives a thread a unique pair number:

```
const int work_index = blockIdx.x * blockDim.x + threadIdx.x;
const int pairs_per_head = head_size / 2;
const int pairs_per_token = projection_size / 2;
const int total_pairs = token_count * pairs_per_token;

if (work_index >= total_pairs) return;
```

`projection_size` is **not** the head size. It is the width of a token's whole Q or K projection. For this model, Q has 32 heads × 64 = `2048` values; K has 8 heads × 64 = `512` values. So Q has 1,024 pairs per token, while K has 256. The launcher uses 256 threads per block and ceiling division to cover the full buffer. The bounds check protects the last partially filled block.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

Next, flattening is undone to find the token, head, and pair:

```
const int token_index = work_index / pairs_per_token;
const int pair_in_token = work_index % pairs_per_token;
const int head_index = pair_in_token / pairs_per_head;
const int pair_index = pair_in_token % pairs_per_head;

const int head_base =
    token_index * projection_size + head_index * head_size;
const int first_index = head_base + pair_index;
const int second_index = first_index + pairs_per_head;
```

For example, with a Q projection, `work_index = 33` is token 0, head 1, pair 1. It rotates columns 65 and 97 of that token's 2,048-value Q buffer: `head_base = 64`, then `64+1` and `64+1+32`. Both belong to the same head.

The position ID chooses the table row, and the pair chooses its column:

```
const int position = position_ids[token_index];
const int table_index = position * pairs_per_head + pair_index;
const float cosine = cos_table[table_index];
const float sine = sin_table[table_index];
```

Finally, the kernel converts the two BF16 inputs to FP32, rotates them, and writes BF16 outputs back **in place**:

```
input[first_index] = __float2bfloat16(x0 * cosine - x1 * sine);
input[second_index] = __float2bfloat16(x0 * sine + x1 * cosine);
```

The FP32 intermediate helps precision in the multiply/add steps. The output is rounded to BF16 because that is this engine's activation format. No other thread writes this pair, so there is no block-wide barrier like RMSNorm needed.

How the host connects RoPE to Q and K

`main` calls `initializeRopeTables` with a 2,048-position limit. `prefill` builds zero-based `position_ids_cpu`, copies them to `position_ids_gpu`, gathers embeddings, and runs the first RMSNorm. At the Part 4 milestone, the RoPE tables and position IDs were ready but `prefill` did not yet call `launchRope`. The current code now applies it to the separate Q and K projection outputs with their respective projection widths. Applying RoPE to `normalized_gpu` would be wrong.

With those projections connected, the mental model is:

```
for each token and each Q/K head:
    for each of the 32 half-split pairs:
        get the token's position and the pair's scaled frequency
        rotate that pair
then attention compares rotated Q against rotated K
```

The dot products are where relative positions show up. The causal mask, grouped-query head mapping, softmax, and mixing of V happen later in attention. RoPE gets the position information into Q/K; it is not the entire attention operation.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

Q, K, V Projection with cuBLAS - Part 5 Video Notes

Video: Computing Query, Key, Value vectors using cuBLAS. Keep `projectBf16RowMajor` and `prefill` in `src/main.cpp` open while reading. RMSNorm explains where the input comes from; RoPE explains what happens to Q and K afterward.

The host snippets describe the Part 5 milestone (`42532af`), before full layers, generation, and batching. Use `git show 42532af:src/main.cpp` for that snapshot. The projection helper's matrix-layout reasoning still applies to the current engine.

Where we are in the engine

Embedding gather gave every token a 2,048-value starting vector. Layer 0's input RMSNorm produced `normalized_gpu`, also `[T, 2048]`, where `T` is the number of prompt tokens. Now we turn each normalized token vector into three **different views of the same current hidden state**:

```
normalized hidden states X [T, 2048]
  ├─ learned Wq -> queries Q [T, 2048] -> 32 heads × 64
  ├─ learned Wk -> keys    K [T,  512] ->  8 heads × 64
  └─ learned Wv -> values  V [T,  512] ->  8 heads × 64
```

The words “query,” “key,” and “value” are names for the jobs these vectors will have in attention. They are **activations calculated for this prompt**. `weights.w_q[0]`, `weights.w_k[0]`, and `weights.w_v[0]` are the fixed learned matrices used to make them.

Why not compare the embedding vectors directly?

My rough notes use “Amazing food” as an intuition. For the token representing “food,” “Amazing” might be useful context, even if the two starting embedding vectors are not especially similar. A plain dot product of raw embeddings only measures one kind of similarity. Attention lets the model learn separate ways to ask a question and to describe what each earlier token offers.

- A **query** is an analogy for what a token is looking for.
- A **key** is an analogy for what a token can be matched on.
- A **value** is the information mixed into the output after the matches are scored.

I say “analogy” because a Q coordinate is not an English-language question, and a K coordinate is not a literal label. The projections are learned numerical transformations. Q and K make attention **directional**: the score for query token `i` looking at key token `j` need not equal the score for query `j` looking at key `i`. Later, the causal mask makes that direction especially important.

The “clean windows” versus “downloaded Windows” example is another way to feel the motivation. Surrounding words can change the useful context for a token. Attention is the mechanism that lets a token combine information from earlier tokens; the Q/K/V projections set up that mechanism. These examples do not imply that any particular head is guaranteed to recognize a specific English relationship.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

What does “projection” mean here?

For one normalized hidden vector `x` with 2,048 values, each output coordinate is a dot product with **one row of a learned weight matrix**:

```
q[o] = sum over h=0..2047 of x[h] * Wq[o, h]
k[o] = sum over h=0..2047 of x[h] * Wk[o, h]
v[o] = sum over h=0..2047 of x[h] * Wv[o, h]
```

For all `T` tokens together, with row-major tensors:

```
Q = X × Wqᵀ       X [T, 2048], Wq [2048, 2048], Q [T, 2048]
K = X × Wkᵀ       X [T, 2048], Wk [ 512, 2048], K [T,  512]
V = X × Wvᵀ       X [T, 2048], Wv [ 512, 2048], V [T,  512]
```

This is a matrix multiplication, not a vocabulary lookup and not yet the dot product between **different tokens**. Each token's row can be projected independently at this stage. The model's printed architecture marks these linear layers `bias=False`, and this helper does not add a bias.

Tiny fake example, just to separate projection from attention: let `x = [2, 1]` and a toy query matrix have rows `[1, 0]` and `[0, 2]`. Then `q = [2, 2]`. A different toy key matrix with rows `[1, 1]` and `[1, -1]` gives `k = [3, 1]` **for the same token**. The matrices choose different learned combinations of the input features. The real model uses 2,048 input features and the trained weight values from SafeTensors.

Why 32 Q heads but only 8 K and V heads?

The Q output width is `2048 = 32 × 64`. K and V each have width `512 = 8 × 64`. Splitting a projected row into consecutive 64-value chunks gives the heads:

```
Q token row: [Q head 0 | Q head 1 | ... | Q head 31]
K token row: [K head 0 | K head 1 | ... | K head 7]
V token row: [V head 0 | V head 1 | ... | V head 7]
```

This is **grouped-query attention** (GQA). Four neighboring query heads share one K head and one V head:

```
Q heads 0-3   -> K/V head 0
Q heads 4-7   -> K/V head 1
...
Q heads 28-31 -> K/V head 7

kv_head = query_head / 4
```

The code sets `QUERY_HEADS_PER_KV_HEAD` from the constants and uses that division in the next attention step. The projection step only creates the differently sized Q, K, and V buffers; it does not copy a K or V head four times. Fewer K/V heads also mean fewer K/V values to cache when decode is implemented.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

The host-side buffers and three calls

`prefill` allocates one GPU buffer for Q and two smaller GPU buffers for K and V:

```
q_bytes  = token_count * HIDDEN_SIZE * sizeof(__nv_bfloat16);
kv_bytes = token_count * KV_DIM      * sizeof(__nv_bfloat16);

cudaMalloc(&q_gpu, q_bytes);
cudaMalloc(&k_gpu, kv_bytes);
cudaMalloc(&v_gpu, kv_bytes);
```

`HIDDEN_SIZE` is `2048`; `KV_DIM` is `512`. For example, with `T = 3`, Q has `3 × 2048` BF16 values, while K and V each have `3 × 512`. The input `normalized_gpu` is already on the GPU from RMSNorm, and the model loader already made direct pointers to the three weight matrices. There is no CPU round trip between the operations.

The host creates a cuBLAS handle and calls the same helper three times:

```
projectBf16RowMajor(handle, normalized_gpu, weights.w_q[0], q_gpu, T, 2048);
projectBf16RowMajor(handle, normalized_gpu, weights.w_k[0], k_gpu, T,  512);
projectBf16RowMajor(handle, normalized_gpu, weights.w_v[0], v_gpu, T,  512);
```

Those lines show the idea; the source uses `cublas_handle`, `token_count`, `HIDDEN_SIZE`, and `KV_DIM` and checks all three returned statuses. The current milestone runs **layer 0 only**. The weight struct has arrays for all 16 layers, but the forward pass has not yet looped over them.

Why use cuBLAS instead of one thread per output?

Unlike embedding gather, every projected output needs a sum over 2,048 input values. With `T` tokens and hundreds or thousands of output columns, that is a large matrix multiplication. cuBLAS supplies tuned GPU GEMM routines for this job. `GEMM` means general matrix multiplication, and `cublasGemmEx` computes:

```
C = alpha * op(A) * op(B) + beta * C
```

Here `alpha = 1`, `beta = 0`, so we want only the new multiplication result. The weight and input are BF16, `CUBLAS_COMPUTE_32F` requests FP32 computation, and the output buffer is BF16. The result is rounded when stored; the whole pipeline is therefore not magically FP32 end to end. NVIDIA's cuBLAS documentation defines the operation, types, and leading dimensions.

The row-major versus column-major puzzle

This is the part that made me stop and draw the matrices. Our buffers are physically **row-major**: a token's features sit next to each other. The classic cuBLAS GEMM interface interprets matrix storage as **column-major**. We do not physically transpose giant GPU buffers just to call it. Instead, we use the fact that the same flat bytes read in column-major look like a transposed matrix.

We want $Y = X \times W^T$, with:

```
X: [T, H]      W: [O, H]      Y: [T, O]
H = 2048       O = 2048 for Q, 512 for K/V
```

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

The row-major result `Y` has the same flat memory layout as a column-major Y^T of shape `[O, T]`. Transposing the equation gives $Y^T = W \times X^T$. In `projectBf16RowMajor`, cuBLAS receives the **weight as A** and the **input as B**:

```
flat W, read column-major, looks like W^T; CUBLAS_OP_T turns it back into W
flat X, read column-major, looks like X^T; CUBLAS_OP_N leaves it as X^T
cuBLAS result: W [O,H] × X^T [H,T] = Y^T [O,T]
flat output bytes: exactly our wanted row-major Y [T,O]
```

So the actual call uses `CUBLAS_OP_T`, `CUBLAS_OP_N`, and `(m, n, k) = (O, T, 2048)`. Its leading dimensions are `lda = 2048` for weights, `ldb = 2048` for input, and `ldc = O` for output. The leading dimension is the distance, in **elements**, between adjacent columns from cuBLAS's point of view; it is not a byte count. The transpose flags describe cuBLAS's view of the buffers, which is why reading just the flags without the storage layout feels backward.

Then RoPE, but only for Q and K

After the three projections, `prefill` calls `launchRope` on `q_gpu` with projection width `2048`, and on `k_gpu` with width `512`. Both use head width `64`, the position IDs, and the precomputed sine/cosine tables. `v_gpu` is left alone. RoPE changes Q/K **activations in place**, so the later Q·K comparisons contain relative-position information.

The host checks cuBLAS return statuses and CUDA launch errors, then synchronizes to catch GPU execution errors. In the Part 5 snapshot, `q_gpu`, `k_gpu`, and `v_gpu` are temporary prefill buffers freed before the function returns. The current engine keeps projection scratch in persistent `InferenceBuffers` for reuse across calls. In this part of the series, the projections' next consumer is the layer-0 self-attention code described in the Part 6 self-attention notes.

What to keep straight while watching

```
Wq/Wk/Wv = fixed learned weight matrices loaded from the model file
Q/K/V    = new prompt-dependent activations produced by matrix multiplication
Q/K      = rotated by RoPE before token-to-token scores
V        = later mixed using attention probabilities, never rotated here
```

The output of this video is a set of three projected buffers. It has not yet decided which earlier token matters most or mixed any values. That is what the attention calculation does next.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

Part 6: Finishing the Self-Attention Sublayer

These notes describe the layer-0 attention implementation shown in Part 6, before the later MLP and decode changes. The source links to `src/main.cpp` and `src/kernels.cu` now point to the newer implementation; use `git show ec066a7:src/main.cpp` and `git show ec066a7:src/kernels.cu` for the Part 6 snapshot. The "Attention" section of `notes.md` gives the working explanation. Start with the Q/K/V projection notes if the three projected buffers are still unfamiliar.

What we are finishing

Now we have `Q`, `K`, and `V` for every prompt token. RoPE has rotated Q and K. How do we get from those vectors to an updated hidden state? This is the big attention calculation people talk about:

```
Q and K -> scaled token-to-token scores
        -> causal mask -> stable softmax -> attention probabilities
probabilities and V -> one weighted value mixture per query head
32 head outputs -> concatenate -> learned output projection
old hidden state + attention update -> new hidden state
```

That sequence completes the **self-attention sublayer** for layer 0 at this point in the series. It does not yet complete the whole Llama decoder layer: the post-attention RMSNorm, gated MLP, and second residual are still to come. Nor does the Part 6 executable run all 16 layers or produce a next token yet.

Why we needed attention in the first place

My notes use "I just clean windows" versus "I just downloaded Windows." The surrounding words help decide what a token is talking about. Attention gives a token a way to pull in information from other tokens in the prompt. In this causal model, a token may use **itself and earlier tokens**, not future ones.

The query/key/value analogy is still useful: a query asks what could help, keys say what can be matched, and values are what gets passed along. But attention weights are learned numerical behavior, not a hand-written English dictionary. A particular head might learn a useful relation, but we do not manually designate "the Windows meaning head."

The full math for one query head

Let `T` be the prompt length. One query head has `Q_h [T, 64]`; the K and V head shared with it each have `[T, 64]`. For query head `h`, let `g = h / 4` be its K/V head. Then:

```
S_h[i,j] = dot(rotated_Q_h[i], rotated_K_g[j]) / sqrt(64) + mask[i,j]
mask[i,j] = 0 if j <= i, and -infinity if j > i

P_h[i,j] = exp(S_h[i,j] - max_allowed_score_in_row)
           / sum_k exp(S_h[i,k] - max_allowed_score_in_row)

head_output_h[i] = sum_j P_h[i,j] * V_g[j]
```

The complete head output is `[T, 64]`. We place 32 of those 64-value chunks beside each other to get `[T, 2048]`, multiply by the learned output matrix `W_o`, and add the result to the incoming hidden state.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

The rough notes wrote the score and mask in two slightly different orders. The code does the **Q·K dot product, scales by** `1 / sqrt(64) = 1/8`**, and then masks future positions**. If an entry is masked to `-infinity`, softmax gives it probability zero. We must not divide a finite mask value by the scale or let a future entry participate in the row maximum.

One worked three-token example

Let's use one tiny fake head and pretend the scores have **already been scaled**. Rows are query positions; columns are key positions. Before masking:

```
             key 0   key 1   key 2
query 0        1       4       3
query 1        2       1       5
query 2        0       1       2
```

The numbers in the upper-right triangle are tempting, but they are future tokens for those queries. Mask them first:

```
             key 0   key 1   key 2
query 0        1      -inf    -inf
query 1        2       1      -inf
query 2        0       1       2
```

Stable softmax turns **each row independently** into probabilities:

```
query 0 -> [1.000, 0.000, 0.000]
query 1 -> [0.731, 0.269, 0.000]
query 2 -> [0.090, 0.245, 0.665]
```

For query 1, subtract the row max `2`, exponentiate `[0, -1, -inf]` to `[1, 0.3679, 0]`, then divide by the sum `1.3679`. That is where `0.731` and `0.269` come from. The mask is about **permission to see a token**; the probability measures **how much its value contributes** among allowed tokens.

Now give the three key positions toy value vectors `V_0 = [10, 0]`, `V_1 = [0, 20]`, and `V_2 = [5, 5]`. Query 1's mixed result is:

```
0.731 * [10, 0] + 0.269 * [0, 20] + 0 * [5, 5]
    ≈ [7.31, 5.38]
```

We have taken information from the **values**, using weights determined by Q and K. The numbers above are teaching examples, not output from this model. Real heads have 64-value vectors, 32 query heads, and BF16 rounding at several storage boundaries.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

The actual shapes and GQA mapping

For this Llama 3.2 1B configuration:

Buffer	Logical shape	What it contains
`q_gpu`	`[T, 32, 64]`, flattened as `[T, 2048]`	Rotated query heads.
`k_gpu`	`[T, 8, 64]`, flattened as `[T, 512]`	Rotated key heads.
`v_gpu`	`[T, 8, 64]`, flattened as `[T, 512]`	Unrotated value heads.
`attention_scores_gpu`	`[32, T, T]`	First scaled scores, then masked scores, then probabilities **in the same allocation**.
`attention_output_gpu`	`[T, 2048]`	The 32 mixed head outputs concatenated by where they are written.
`output_projection_gpu`	`[T, 2048]`	Learned output projection, ready for residual addition.

Head mapping is `kv_head_index = query_head_index / 4`: Q heads 0-3 share K/V head 0, and so on up to Q heads 28-31 sharing K/V head 7. Each Q head still has its **own** score and probability matrix. Sharing K/V heads reduces the amount of K/V data; it does not mean all four query heads must look at the same token.

The score tensor's storage grows as `32 × T × T`. At `T = 2048`, BF16 scores alone occupy `32 × 2048² × 2 = 268,435,456` bytes, or 256 MiB. This straightforward prefill path materializes those matrices. A later optimized attention implementation could avoid some of that memory, but this code is useful because every step is visible.

Step 1: scaled Q·K scores with cuBLAS

`computeGqaAttentionScores` loops over all 32 query heads. For head `h`, it takes a pointer to that 64-value Q slice and a pointer to K head `h / 4`. The vectors are physically inside token-major rows: Q rows have stride `2048`, K rows have stride `512`. Passing `q + h * 64` or `k + (h/4) * 64` changes the **head offset**, but not the distance to the next token's row.

For a query token `i` and key token `j`, the desired score is:

```
score[h, i, j] = sum over d=0..63 of Q[i,h,d] * K[j,h/4,d] / 8
```

`alpha = 1 / sqrt(HEAD_DIM)` supplies that `/ 8` during `cublasGemmEx`; `beta = 0` means the old score buffer is ignored. The GEMM's logical result for each head is `[T, T]`. Because the buffers are row-major but classic cuBLAS uses a column-major view, the call uses `CUBLAS_OP_T` for K and `CUBLAS_OP_N` for Q. `lda = KV_DIM` and `ldb = HIDDEN_SIZE` preserve the full token strides even though we selected only 64 columns of each row. `ldc = T` makes each head's score matrix contiguous.

One clarification from my rough notes: negative dot products are perfectly valid scores. We do **not** run softmax because negative numbers are somehow meaningless. We run it because attention needs nonnegative weights, normalized **across the allowed key positions** for each query row. Scaling first also keeps score magnitudes from growing with the 64 summed dimensions and pushing softmax too sharply toward one key.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

cuBLAS takes BF16 Q/K values, uses `CUBLAS_COMPUTE_32F`, then stores BF16 scores. FP32 compute helps the dot product, but storing scores as BF16 still rounds them before the next kernel.

Step 2: make future positions impossible to use

`causalMaskKernel` launches one thread per score element in flattened `[head, query, key]` storage. It recovers the two token indices with:

```
key_token = score_index % token_count;
query_token = (score_index / token_count) % token_count;
```

When `key_token > query_token`, the thread writes negative infinity. The diagonal remains valid, so every query row always has **at least its own token** to attend to. This is why the later softmax always has at least one finite candidate. A query at position 0 may use only key 0; a query at position 2 may use keys 0, 1, and 2.

The score matrix is not symmetric after masking. That is expected: the token at position 2 can use position 0, while position 0 cannot peek at position 2. RoPE encodes position in Q/K; the causal mask enforces this one-way information flow.

Step 3: stable softmax, one row at a time

`stableSoftmaxKernel` launches **one CUDA block per** `(query head, query token)` **row**, with 256 threads in that block. Every thread visits key positions `thread_index`, `thread_index + 256`, and so on. That stride lets the same kernel process a row longer than 256 keys.

It makes two shared-memory reductions:

1. Find the **maximum** score in the row, ignoring nothing explicitly because masked values are already `-infinity`.
2. Sum `exp(score - row_max)` across all key positions.

Then each thread writes `exp(score - row_max) / sum` to the matching location in the same `scores` buffer. Subtracting the maximum does not change softmax probabilities, because the same constant is subtracted from every allowed score, but it prevents very large positive scores from overflowing `exp`.

The reductions and exponentials use FP32. The probabilities are stored as BF16, so their stored row sum may be **approximately**, rather than exactly, 1. A masked `-infinity` gives `exp(-infinity) = 0`, so future positions have zero probability. `launchCausalMask` runs before `launchStableSoftmax` in `prefill`; reversing those calls would be wrong.

Step 4: use probabilities to mix V

`mixGqaValues` loops over 32 query heads. It selects that head's `[T, T]` probability matrix and its shared V head `h / 4`, then performs a cuBLAS matrix multiplication:

```
head_output_h [T, 64] = P_h [T, T] × V_(h/4) [T, 64]
```

For one query token `i` and one output feature `d`, that is `sum_j P_h[i,j] * V_(h/4)[j,d]`. Future `j` values contribute nothing because their probabilities are zero. Values are **not** passed through RoPE: Q/K choose what to take; V supplies what gets taken.

Again, the cuBLAS arguments look strange until you remember the row-major storage. Both operands use `CUBLAS_OP_N` in cuBLAS's column-major view to produce the flat bytes of the wanted row-major `P × V`. The V

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

token stride is still `KV_DIM = 512`; each probability row has stride `T`; and the destination token stride is `HIDDEN_SIZE = 2048`.

The code points `output_head` at `attention_output_gpu + h * 64`. That is how the 32 results land side by side inside each token's 2,048-value row. We do not need a separate concatenate kernel:

```
token i output row = [head 0's 64 values | head 1's 64 values | ... | head 31's 64 values]
```

Step 5: output projection and residual connection

The concatenated head output is not yet the attention update in the model's hidden space. `prefill` calls the same `projectBf16RowMajor` helper used for Q/K/V, this time with `weights.w_o[0]` and output width `2048`:

```
attention_update = attention_output × Woᵀ     [T, 2048]
```

`W_o` is learned. It mixes features from the different heads and produces a vector with the same width as the incoming hidden state, so they can be added. The CUDA `residualAddKernel` gives one thread each hidden value, converts the two BF16 inputs to `float`, adds them, and writes BF16 back **in place**:

```
activations_gpu[token, feature] += output_projection_gpu[token, feature]
```

My Δx picture from the rough notes is good here: the attention path computes an update, and the residual adds it to the old vector. The subtle part is **which old vector**. This is a pre-norm layer: RMSNorm made `normalized_gpu` for Q/K/V, but the skip path uses the original layer input in `activations_gpu`. For layer 0 that input came from embedding gather. In later layers it would be the previous layer's hidden state. Adding to `normalized_gpu` instead would implement a different computation.

How this appears in `prefill`

The source order is worth tracing with your finger:

```
embedding gather -> activations_gpu
RMSNorm -> normalized_gpu
project Wq, Wk, Wv -> q_gpu, k_gpu, v_gpu
RoPE -> modifies q_gpu and k_gpu
computeGqaAttentionScores -> attention_scores_gpu contains scaled scores
launchCausalMask -> future scores become -infinity
launchStableSoftmax -> same buffer now contains probabilities
mixGqaValues -> attention_output_gpu
project Wo -> output_projection_gpu
launchResidualAdd -> modifies activations_gpu
```

The host checks cuBLAS statuses and CUDA launch results along the way, then synchronizes to catch GPU execution errors. The temporary buffers are freed at the end of `prefill`. In the Part 6 version, the updated `activations_gpu` is also freed rather than passed into a post-attention norm or another layer. That version demonstrates this attention computation, but does not yet generate text.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

What still makes a full Transformer layer

After the attention residual, Llama's decoder layer continues with a second RMSNorm and a gated feed-forward network: gate and up projections, SiLU on the gate, elementwise gate/up multiplication, down projection, and another residual add. Then the next layer repeats the whole sequence. The Part 6 source has pointers to those MLP weights but does not execute that path. Part 7 completes it and predicts the first next token.

If you remember one line from this note, use this one:

```
Attention = use rotated Q/K to decide *where to look*,
            use probabilities to mix V from those positions,
            project the mixture, then add it to the incoming hidden state.
```

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

Part 7: Feed-Forward Network, SwiGLU, and the First Predicted Token

These are companion notes for the Part 7 video. They develop the feed-forward section of my working notes into a code walkthrough. Part 6 finished self-attention; this part finishes each Llama layer, runs all 16 layers, and selects one next token. Part 8 later adds cached, token-at-a-time generation.

The code excerpts below use the **prompt prefill that predicts one next token**, shown in the video and committed as `e9399f9`. Some excerpts show the calls without their surrounding error checks so the data flow is easier to follow. The current `src/main.cpp` also has the later KV-cache, decode, batching, and paging paths. To see the complete video-era source rather than the later additions, run `git show e9399f9:src/main.cpp` and `git show e9399f9:src/kernels.cu`.

Where this work fits

For one token position, let `h` be the hidden state entering a decoder layer. Attention has already mixed in information from the allowed earlier positions:

```
a = h + Attention(RMSNorm(h))
x = RMSNorm(a)                          // second, post-attention norm
gate = x × W_gateᵀ                      // 2048 -> 8192
up   = x × W_upᵀ                        // 2048 -> 8192
m    = SiLU(gate) ⊙ up                  // elementwise, still 8192
down = m × W_downᵀ                      // 8192 -> 2048
next_hidden = a + down                  // second residual
```

The same sequence runs for every token position and then repeats with **different weights** in layers 0 through 15. After layer 15, the last prompt position goes through a final RMSNorm, the language-model head, and argmax. This video ends after predicting **one** next token; it does not yet loop to generate a response.

Why have a feed-forward network after attention?

An analogy from my rough notes: imagine each token as a person holding a page of numbers. Attention is the round in which people can exchange information with permitted earlier people. The feed-forward network, or **FFN**, is the next round: each person applies the same kind of learned processing to *their own updated page*. The FFN does not perform another exchange between token positions.

That does **not** mean the FFN's input has no context. Attention has already put context from earlier tokens into each updated hidden vector. The FFN processes those context-aware features one position at a time. In code, using one matrix multiplication for all prompt rows is a faster way to apply the same layer's weights independently to every row; it does not mix rows together.

The weights were learned during training. During this inference run we read them and calculate outputs; we are not adjusting them to make the answer better. A different token vector produces different gate and up values even though this layer's weights stay fixed.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

Build the idea from one perceptron

A simple perceptron combines inputs with weights, adds a bias, and applies a decision rule. Suppose an imaginary urgency detector gets `x[0] = 1` when a message contains “urgent” and `x[1]` equal to its exclamation-mark count:

```
float score = 1.5f * x[0] + 0.25f * x[1] - 1.0f;
bool is_urgent = score > 0.0f;
```

For `x = [1, 2]`, the score is `1.5 + 0.5 - 1 = 1`, so the toy detector says yes. The weights decide how much each input contributes; the `-1` is a **bias** that moves the decision boundary. These values are hand-chosen for the example. Real model weights are learned.

Remove the yes/no threshold for a moment and imagine thousands of different weighted sums of the **same** input. Put each sum's weights in one row of a matrix. A matrix multiplication calculates all those sums and gives us a new vector. This is a **linear projection**. The projections in this Llama MLP produce numerical features, not urgent/not-urgent booleans, and they have **no bias**.

A multilayer perceptron, or **MLP**, puts learned projections and nonlinear operations together. Nonlinearity matters: if we only applied one linear matrix and then another, their product could be replaced with one matrix. SiLU and the multiplication of two branches let the MLP represent relationships that a single linear projection cannot.

Llama's MLP has a particular gated shape. The gate and up projections are **parallel**: both read the same normalized input `x`. We do not feed the up output into the gate projection or the gate output into the up projection. Only after both exist do we combine their values.

Follow the shapes before the code

Let `T` be the number of prompt tokens. A weight matrix is listed as `[output features, input features]` because each output feature owns one row of weights.

Item	Shape	Purpose
`activations_gpu` after attention residual	`[T, 2048]`	Hidden state to preserve for the MLP skip connection.
`normalized_gpu`	`[T, 2048]`	Post-attention RMSNorm output supplied to both branches.
`W_gate`, `W_up`	`[8192, 2048]` each	Two different learned views of the same input.
`gate_gpu`, `up_gpu`	`[T, 8192]` each	One gate/up value per token and intermediate feature.
`gate_gpu` after SwiGLU	`[T, 8192]`	`SiLU(gate) * up`, written over the old gate values.
`W_down`	`[2048, 8192]`	Learned mixing back to hidden width.
`output_projection_gpu` after down projection	`[T, 2048]`	MLP update to add to `activations_gpu`.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

For a **single** token, the up projection is `[1, 2048] × [2048, 8192] = [1, 8192]`. Each of the 8,192 answers is a dot product between the 2,048-value input and one row of `W_up`. The gate projection repeats that shape with its own weights. “Up” means more intermediate features, not copying values or padding with zeros. “Down” uses learned weights to mix the 8,192 values back to 2,048; it does not simply discard the extra positions.

After the MLP residual, every token still has a 2,048-value hidden vector. That unchanged outer shape is why the result can be passed into the next decoder layer.

What SiLU and the gate actually do

For one gate value `g`:

```
sigmoid(g) = 1 / (1 + e^(-g))
SiLU(g)    = g * sigmoid(g) = g / (1 + e^(-g))
output[j]  = SiLU(gate[j]) * up[j]
```

`sigmoid(g)` lies between zero and one, but **SiLU(g) need not**. It may be negative or greater than one. So “gate” is an analogy, not a strict binary switch. For the small examples in my working notes:

```
gate                = [ 2,      -2,       0]
up                  = [ 3,       4,       5]
SiLU(gate)          ≈ [ 1.7616, -0.2384, 0]
SiLU(gate) ⊙ up     ≈ [ 5.2848, -0.9536, 0]
```

The first up value is amplified, the second changes sign and becomes smaller in magnitude, and the third contributes zero. The real vector has 8,192 entries. ⊙ means **elementwise multiplication**: entry 0 times entry 0, entry 1 times entry 1, and so on. It is not a dot product, so the result remains 8,192 values. This SiLU-and-multiply arrangement is the **SwiGLU** step in this implementation.

SiLU is smooth, which helped make these weights trainable with gradients. The inference code only evaluates the learned function; it does not calculate gradients or update weights.

Code: locate each layer's weights

The loader keeps direct GPU pointers for every layer's MLP matrices. The following assignments appear in the model loader in `src/main.cpp`:

```
weights.mlp_gate_proj[layer] = reinterpret_cast<__nv_bfloat16 *>(
    base + offsets.at(prefix + ".mlp.gate_proj.weight"));
weights.mlp_up_proj[layer] = reinterpret_cast<__nv_bfloat16 *>(
    base + offsets.at(prefix + ".mlp.up_proj.weight"));
weights.mlp_down_proj[layer] = reinterpret_cast<__nv_bfloat16 *>(
    base + offsets.at(prefix + ".mlp.down_proj.weight"));
```

`base` points at the model data already copied to GPU memory. `offsets.at(...)` finds the byte offset recorded in the safetensors header. `reinterpret_cast` makes a typed BF16 pointer at that location; these lines do not run the MLP or copy another matrix. The layer number becomes part of `prefix`, so layer 0 and layer 1 point at different learned tensors. Within one layer, all prompt positions use that same layer's tensors.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

Code: normalize, then compute both branches

In video-era `prefill`, `gate_gpu` and `up_gpu` are allocated as `T × 8192` BF16 elements **before** the 16-layer loop and reused in each layer. After the attention projection has been added to `activations_gpu`, the MLP path starts:

```
launchRmsNorm(
    activations_gpu,
    normalized_gpu,
    weights.post_attn_layernorms[layer],
    token_count);

const cublasStatus_t gate_status = projectBf16RowMajor(
    cublas_handle, normalized_gpu, weights.mlp_gate_proj[layer],
    gate_gpu, token_count, HIDDEN_SIZE, INTERMEDIATE_SIZE);

const cublasStatus_t up_status = projectBf16RowMajor(
    cublas_handle, normalized_gpu, weights.mlp_up_proj[layer],
    up_gpu, token_count, HIDDEN_SIZE, INTERMEDIATE_SIZE);
```

`launchRmsNorm` reads `activations_gpu` and writes a separate `normalized_gpu`. We covered the RMSNorm calculation in Part 3; the important detail here is **which buffer survives**. `activations_gpu` retains the hidden state from after the attention residual, ready for the second residual later. Both projection calls read `normalized_gpu`. Their different weight pointers and different output buffers make them parallel branches, although these host calls are issued in sequence.

`projectBf16RowMajor` is the same cuBLAS helper used for Q/K/V. Its last three arguments are the token count, input width, and output width. Here they are `T`, `2048`, and `8192`. Conceptually it computes `input × weightᵀ` in row-major layout. The central cuBLAS arguments are:

```
CUBLAS_OP_T, CUBLAS_OP_N,
output_features, token_count, input_features,
&alpha, weight, CUDA_R_16BF, input_features,
input, CUDA_R_16BF, input_features,
&beta, output, CUDA_R_16BF, output_features,
CUBLAS_COMPUTE_32F, CUBLAS_GEMM_DEFAULT
```

The output width is cuBLAS's `m`, the token count is `n`, and the input width is the shared reduction size `k`. `CUBLAS_OP_T` on the weight and `CUBLAS_OP_N` on the input account for cuBLAS interpreting the same row-major bytes as column-major matrices; no GPU buffer is physically transposed. `alpha = 1` applies the product, and `beta = 0` overwrites old scratch contents. The multiplication accumulates with `CUBLAS_COMPUTE_32F`, then stores these branch outputs as BF16. The two `cublasStatus_t` results are checked so a failed projection does not silently continue.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

Code: one CUDA thread per SwiGLU value

`swiGluKernel` implements the formula literally:

```
const size_t index = static_cast<size_t>(blockIdx.x) * blockDim.x + threadIdx.x;
if (index >= value_count)
{
    return;
}

const float g = __bfloat162float(gate[index]);
const float u = __bfloat162float(up[index]);
gate[index] = __float2bfloat16((g / (1.0f + expf(-g))) * u);
```

`blockIdx.x * blockDim.x + threadIdx.x` gives one flat index across all `T × 8192` values. The bounds check protects the last CUDA block when the count is not a multiple of 256. The two BF16 inputs become `float` for the exponential and multiply. `g / (1 + expf(-g))` is SiLU. Multiplying by `u` produces the gated value, then `__float2bfloat16` rounds it back to BF16.

The write goes into `gate[index]` **in place**. After this kernel, `gate_gpu` no longer contains raw gate scores; it contains the MLP intermediate `SiLU(gate) ⊙ up`. The up buffer is read but not modified. The launcher uses 256 threads per block and enough blocks to cover all `T × 8192` entries. This is an elementwise kernel: a thread needs no values from another token or another intermediate feature.

Code: project down and add the correct residual

The host then uses the modified `gate_gpu` as the down projection input:

```
launchSwiGlu(gate_gpu, up_gpu, token_count, INTERMEDIATE_SIZE);
projectBf16RowMajor(
    cublas_handle,
    gate_gpu,
    weights.mlp_down_proj[layer],
    output_projection_gpu,
    token_count,
    INTERMEDIATE_SIZE,
    HIDDEN_SIZE);
launchResidualAdd(
    activations_gpu,
    output_projection_gpu,
    token_count,
    HIDDEN_SIZE);
```

The down projection multiplies `[T, 8192]` by the transpose of `[2048, 8192]` and writes `[T, 2048]`. `output_projection_gpu` held the attention projection earlier, but that value has already been added to `activations_gpu`, so its allocation can now hold the MLP projection. Reusing the buffer avoids another `T × 2048` allocation.

`launchResidualAdd` modifies `activations_gpu` in place: `activations_gpu += output_projection_gpu`. Its left side is the hidden state **from before the second RMSNorm**, after the attention residual. Adding the MLP result to `normalized_gpu` would change the model. The code groups these calls under error checks and synchronizes afterward to catch a GPU execution failure before starting the next layer.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

Code: repeat the whole decoder layer 16 times

The video-era prefill wraps the original attention path and the new MLP path in:

```
for (int layer = 0; layer < N_LAYERS; ++layer)
{
    // Input norm -> Q/K/V -> RoPE -> causal GQA -> output projection.
    // Attention residual -> post-attention norm -> gate/up -> SwiGLU.
    // Down projection -> MLP residual.
}
```

`N_LAYERS` is 16. This is one complete forward pass through the same **sequence of operations**, not 16 copies of the same learned weights. The arrays `weights.w_q[layer]`, `weights.post_attn_layernorms[layer]`, `weights.mlp_gate_proj[layer]`, and the other layer-indexed pointers select that layer's tensors. The same scratch buffers can be reused because only the completed `activations_gpu` must pass to the next layer. Its shape stays `[T, 2048]`, while its numerical contents change at each layer. We do not select a token after each layer.

Code: from the final hidden vector to vocabulary scores

After layer 15, only the **last prompt position** is needed to predict what follows the prompt. Causal attention lets that position incorporate itself and all earlier prompt positions. The video-era code selects its 2,048-value row and applies the final RMSNorm:

```
const __nv_bfloat16 *last_hidden =
    activations_gpu + static_cast<size_t>(token_count - 1) * HIDDEN_SIZE;
launchRmsNorm(last_hidden, normalized_gpu, weights.norm, 1);
```

The offset skips `token_count - 1` complete hidden rows. Passing `1` normalizes just that row, so we do not spend work projecting every prompt position into vocabulary scores when we only need the first response token.

The **LM head** (“language-model head”) is a matrix with logical shape `[128256, 2048]`: one 2,048-weight row per vocabulary token. Its dot product with the normalized last hidden row produces **128,256 logits**, one score for each possible next token:

```
logit[token_id] = sum over feature=0..2047 of
                  normalized_last_hidden[feature] * LM_head[token_id, feature]
```

The loader uses `lm_head.weight` if the safetensors file provides it; otherwise it points `weights.lm_head` at `weights.embed_tokens`. That fallback is **weight tying**: the same learned table serves both input embeddings and output scoring. It is still an output projection, but we should not assume every checkpoint stores a separate LM-head tensor.

The code calls cuBLAS with `VOCAB_SIZE, 1, HIDDEN_SIZE` as its output rows, token columns, and reduction width:

```
cublasGemmEx(
    cublas_handle, CUBLAS_OP_T, CUBLAS_OP_N,
    VOCAB_SIZE, 1, HIDDEN_SIZE,
    &alpha, weights.lm_head, CUDA_R_16BF, HIDDEN_SIZE,
    normalized_gpu, CUDA_R_16BF, HIDDEN_SIZE,
    &beta, logits_gpu, CUDA_R_32F, VOCAB_SIZE,
    CUBLAS_COMPUTE_32F, CUBLAS_GEMM_DEFAULT);
```

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

The input and weights are BF16, the GEMM computes in FP32, and `logits_gpu` stores FP32. These are **scores**, not probabilities: a logit of `5` does not mean a 50% chance. Higher logits rank ahead of lower logits for this one prediction.

Code: argmax returns a token ID, not its score

My rough-note example uses a five-token vocabulary:

```
token ID:    0     1     2     3     4
logit:      0.4  -1.1   3.7   2.0  -0.3
```

The largest **value** is `3.7`; its **index** is token ID `2`. Argmax returns `2`. For the real 128,256-entry logit vector, `argmaxKernel` launches one block of 256 threads. Each thread scans indices `threadIdx.x`, `threadIdx.x + 256`, and so on, keeping its best score and ID. The kernel then reduces the 256 candidates through shared memory:

```
for (int index = thread; index < vocabulary_size; index += THREADS)
{
    const float value = logits[index];
    if (value > best_value || (value == best_value && index < best_index))
    {
        best_value = value;
        best_index = index;
    }
}
best_values[thread] = best_value;
best_indices[thread] = best_index;
__syncthreads();
```

At this vocabulary size, each thread examines 501 entries. The shared-memory reduction then halves the candidate count on every pass:

```
for (int stride = THREADS / 2; stride > 0; stride >>= 1)
{
    if (thread < stride)
    {
        const float other_value = best_values[thread + stride];
        const int other_index = best_indices[thread + stride];
        if (other_value > best_values[thread] ||
            (other_value == best_values[thread] && other_index < best_indices[thread]))
        {
            best_values[thread] = other_value;
            best_indices[thread] = other_index;
        }
    }
    __syncthreads();
}
```

The comparison chooses the **smaller token ID** when scores tie, so argmax is deterministic. Thread 0 writes the winning ID to `next_token_gpu`; `cudaMemcpy` brings the single integer back to the CPU. In the video-era `main`, the final print is:

```
std::cout << "Next token ID: " << next_token_id << '\n';
std::cout << "Next token: " << tokenizer.decode({next_token_id}) << '\n';
```

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

`tokenizer.decode` turns the ID into its text or bytes. One token might be a whole word, part of a word, punctuation, or another token piece.

This is **greedy** selection. We do not need a vocabulary softmax first: exponentiation and division by the same positive sum preserve the ordering of finite logits, so the largest logit would also have the largest softmax probability. Attention's softmax has a different job: it produces weights for mixing value vectors.

Keep these distinctions straight

- The attention output projection `W_o` is a **weight matrix**. Its computed output, after adding the attention residual, is the hidden state that enters the second norm.
- The FFN does no *new* cross-token mixing. Its input can already contain context gathered by attention.
- Gate and up are two projections from the **same** normalized hidden vector. Their outputs are multiplied elementwise after SiLU.
- `sigmoid(g)` is between zero and one; the full `SiLU(g) * up` result can amplify a value, change its sign, or make it zero.
- The MLP residual adds to the hidden state **before** the post-attention RMSNorm, not to that norm's output.
- A complete layer preserves hidden width 2,048. All 16 layers use their **own** learned weights.
- The LM head produces logits. Argmax chooses the **token ID** at the highest logit; the tokenizer turns that ID into readable output.
- The Part 7 code predicts one token from prefill. Part 8 explains how subsequent predicted tokens are fed back through the model.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

Part 8: Prefill, Decode, and the KV Cache

These notes accompany Part 8: Decode and KV cache. They expand the decode section of my working notes into a guide to the ideas, shapes, and implementation. Part 7 finished the transformer and predicted one next token. This part explains how that becomes a response made of many tokens.

The video-era implementation is commit `7561242`. It handles one request with contiguous key and value allocations. The current source adds batching, persistent scratch buffers, a shared paged cache, and direct paged decode attention. The underlying token timeline is the same. Sections below identify which implementation they describe so the later code does not obscure the lesson.

Read the full video-era files with:

```
git show 7561242:src/main.cpp
git show 7561242:src/kernels.cu
```

For the current implementation, keep `src/main.cpp`, `src/kernels.cu`, and `include/kernels.cuh` open alongside these notes. This engine targets a fixed Llama 3.2 1B Instruct layout; the dimensions below come from its source rather than from a generic model configuration.

1. Two phases of one generation process

Prefill processes the prompt tokens. All of them are already known, so a layer can use matrix operations with one row per prompt token. Causal attention still prevents a position from using a future position.

Decode repeatedly processes the latest generated token to predict another one. For one sequence, only one new input token is available per iteration. That token still goes through every transformer layer.

The word “decode” also appears in the tokenizer interface. `Tokenizer::decode` turns IDs into text. The inference `decode` function performs a transformer forward pass. These are different operations.

For a concrete timeline, let each letter stand for one token ID:

```
Prompt:                   [A, B, C]
Prefill processes:         [A, B, C]       -> predicts D
First decode processes:             D     -> predicts E
Second decode processes:               E  -> predicts F
```

Predicting `D` selects an integer from the vocabulary. It does not run `D` through the transformer. `D` needs its own embedding lookup and forward pass before it can predict `E`.

Completed operation	Tokens processed and cached	Latest predicted ID	Next input position
Prefill	`A, B, C`	`D`	3
Decode `D`	`A, B, C, D`	`E`	4
Decode `E`	`A, B, C, D, E`	`F`	5

Positions are zero based. The cache length counts processed tokens, so it is also the position assigned to the next token we process. The newly predicted ID is one step ahead of the cache.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

2. Why earlier positions can be reused

One way to generate would be to run the entire growing prefix again:

```
forward(A, B, C)       -> D
forward(A, B, C, D)    -> E
forward(A, B, C, D, E) -> F
```

That repeats the embeddings, normalizations, projections, attention, and MLP work for `A`, `B`, and `C` every time. Causality gives us a reason to avoid that repetition.

Before appending `D`:

```
A can attend to A
B can attend to A, B
C can attend to A, B, C
```

After appending `D`, those three sets stay the same. Only the new position gets a new attention calculation:

```
D can attend to A, B, C, D
```

For layer 0, earlier positions receive the same embeddings, use the same fixed weights, keep the same absolute positions, and attend to the same allowed prefix. RMSNorm and the MLP process each position independently. Their earlier-position outputs therefore do not need to change.

Layer 1 receives those unchanged earlier-position outputs, so the same reasoning applies again. Following this argument through all 16 layers explains why appending a token does not require recomputing the earlier tokens.

This reasoning assumes the same model, prefix, positions, and inference computation. It does not make an arbitrary cached occurrence of a word reusable in a different prompt: the hidden states and later-layer K/V depend on that word's earlier context.

3. What the cache contains

At every layer, future attention needs earlier **keys and values**. We keep those activation vectors after computing them once.

Item	Stored for future tokens?	Reason
Rotated K vectors	Yes	Future queries compare against these keys.
V projection outputs	Yes	Future attention combines these values.
Earlier Q vectors	No	A future position calculates its own query.
Attention scores or probabilities	No	A different query produces different scores.
`W_k` and `W_v`	Model weights, separate from the cache	These fixed matrices produce the sequence-dependent vectors.
Every intermediate hidden state	No	Future decode attention only needs the earlier K/V from each layer.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

The key cache stores **K after RoPE**. The value cache stores **V without RoPE**. Reading an old key does not rotate it again.

Each layer needs its own cache. Layer 5 has different weights and receives different hidden states from layer 0, so layer 0's keys cannot stand in for layer 5's keys.

For one layer after processing `A, B, C`, the logical view is:

```
K: [k_A] [k_B] [k_C]     each row contains 512 BF16 values
V: [v_A] [v_B] [v_C]     each row contains 512 BF16 values
```

Each row contains eight 64-value heads placed beside one another. Query heads are grouped in fours: query heads `0..3` use KV head `0`, query heads `4..7` use KV head `1`, and so on. Integer division implements this mapping:

```
kv_head = query_head / 4
```

We retain eight heads of K/V, not 32 duplicated copies to match the query count. That is one memory benefit of grouped-query attention, or GQA.

4. Track the tensor shapes

Let `T` be the prompt length and `L` the number of tokens already cached before a decode step. The table describes one sequence. In current batched decode, the leading `1` becomes the number of active sequences for projections and MLP operations; each sequence still has its own context length.

Tensor or operation	Prefill	One decode step
Hidden state	`[T, 2048]`	`[1, 2048]`
Q	`[T, 2048] = [T, 32, 64]`	`[1, 2048]`
New K and new V, each	`[T, 512] = [T, 8, 64]`	`[1, 512]`
One layer's valid K or V after writing	`[T, 512]`	`[L + 1, 512]`
Scores for one query head, conceptually	`[T, T]`	`[1, L + 1]`
Attention output across all heads	`[T, 2048]`	`[1, 2048]`
MLP gate and up outputs, each	`[T, 8192]`	`[1, 8192]`
Vocabulary logits used for prediction	`[1, 128256]`	`[1, 128256]`

Prefill needs the hidden states of every prompt token through all layers: later tokens attend to the K/V computed for earlier ones. After the final layer, this generation path only uses the last prompt position for the next-token logits. It does not need a vocabulary projection for every prompt row.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

For `L = 3`, one head's decode attention is:

```
q_D_head × K_cached_headᵀ / sqrt(64)
[1, 64] × [64, 4] -> [1, 4] scores

softmax(scores) × V_cached_head
[1, 4] × [4, 64] -> [1, 64] head output
```

The four positions are `A`, `B`, `C`, and `D`. All are allowed by causality. There is no future column to mask in this one-token decode calculation, but reading only the valid prefix is still essential. An allocated slot is not automatically valid context.

5. One decode step, from ID to next prediction

Suppose prefill has processed `A, B, C` and predicted `D`. Before decode, `cache.length == 3`.

1. Look up `D`'s embedding to produce `[1, 2048]`.
2. Assign absolute position `3` to this token.
3. In layer 0, normalize and project the new hidden row into Q, K, and V.
4. Rotate the new Q and K using RoPE position `3`.
5. Write the new K/V into this layer's cache at position `3`.
6. Attend over positions `0..3`, including `D` itself.
7. Apply the output projection, attention residual, second RMSNorm, SwiGLU MLP, and MLP residual.
8. Repeat steps 3 through 7 for layers 1 through 15, using each layer's weights and cache.
9. Apply the final RMSNorm and LM head, then choose the largest logit.
10. After a successful pass, increase the cache length to `4`.

If argmax predicts `E`, the cache now holds `A, B, C, D`. `E` has no cached K/V yet. It becomes the next decode input if generation continues.

The sequence position stays `3` throughout all 16 layers. Entering another layer does not advance the sequence. Incrementing the length inside the layer loop would write different layers into different token slots.

6. RoPE needs the sequence position

The new hidden-state buffer has just one row. Its row index is `0`; its sequence position is `L`. These numbers have different jobs.

Current `decode` constructs positions with:

```
position_ids_cpu[row] = cache->length;
```

For a single sequence with three cached tokens, that uploads `[3]`. Later:

```
launchRope(q_gpu, position_ids_gpu, cos_table_gpu, sin_table_gpu,
           token_count, HIDDEN_SIZE, HEAD_DIM);
launchRope(k_gpu, position_ids_gpu, cos_table_gpu, sin_table_gpu,
           token_count, KV_DIM, HEAD_DIM);
```

These excerpts omit error handling to show the data flow. `token_count` is the number of active decode rows in the current implementation; it is `1` for this example.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

Inside `ropeKernel`, `token_index` locates the input row and `position_ids[token_index]` selects the correct sine/cosine table row. Each 64-value head pairs feature `i` with feature `i + 32`, as explained in Part 4.

The previously cached keys already carry their original positions. Reusing them alongside the newly rotated key preserves the positional comparison. Resetting every decode token to position zero, or rotating old keys again, changes that comparison.

7. Video-era code: a contiguous cache

At commit `7561242`, `KVCache` is deliberately simple:

```
struct KVCache
{
    __nv_bfloat16 *keys = nullptr;
    __nv_bfloat16 *values = nullptr;
    int capacity = 0;
    int length = 0;
};
```

Each allocation has logical shape `[16, capacity, 512]`. `initializeKvCache` allocates all those rows up front. `capacity` is reserved space; `length` says how much contains usable results.

The offset to one layer and a new position is:

```
const size_t layer_offset =
    static_cast<size_t>(layer) * cache.capacity * KV_DIM;
__nv_bfloat16 *layer_keys = cache.keys + layer_offset;
__nv_bfloat16 *new_key =
    layer_keys + static_cast<size_t>(position) * KV_DIM;
```

These are offsets in BF16 **elements**, not bytes. The V allocation uses the same indexing.

Prefill projects prompt K/V into temporary buffers, rotates K, and copies those rows into each layer's cache. The valid length becomes `T` after the successful prefill.

Video-era decode instead projects the new K/V directly into their cache row. It rotates `new_key` in place and then runs:

```
computeGqaAttentionScores(..., query_count=1, key_count=L+1)
launchStableSoftmax(..., query_count=1, key_count=L+1)
mixGqaValues(..., query_count=1, key_count=L+1)
```

`computeGqaAttentionScores` and `mixGqaValues` accept a separate key count so attention can be rectangular: one query row against many key rows. The stored score/probability buffer has shape `[32, 1, L + 1]`. No decode causal-mask launch is needed because the prefix contains no future tokens.

That version creates and frees scratch allocations and a cuBLAS handle inside each forward call. Those are temporary working buffers; the K/V allocations survive from prefill until the request finishes. Keeping this distinction clear explains how cached context persists even when local decode scratch is freed.

8. Current code: the same logical cache in pages

The current `KVCache` holds a `KVCachePool*`, a `KVBlockTable`, a capacity, and a valid length. It no longer owns contiguous `keys` and `values` pointers.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

One physical page contains K and V for **16 token positions in one layer**:

```
physical page:
[16 × 512 K values][16 × 512 V values]
```

A sequence's block table maps `[layer][logical_page]` to a physical page number in the shared pool. For logical position `p`:

```
logical_page   = p / 16
offset_in_page = p % 16
physical_page  = cache.block_table.layers[layer][logical_page]
```

At position `15`, the new row fits in the first logical page. At position `16`, every layer needs its next page. The physical pages need not be adjacent in memory.

`KVPageAllocator::reserve` checks how many missing pages all layers need before changing the table. Prefill reserves its prompt pages. Decode reserves pages through `length + 1` before writing or uploading the block tables.

`appendKvCache` copies rows from projection scratch into the mapped pages. It handles a partially filled page and crossings between pages, so the same helper supports a whole prompt and a single decode row. It does not increment `length`; the successful outer forward pass commits that update.

`freeKvCache` now returns mapped pages to the allocator and resets the valid length. The shared allocation remains alive for other requests. Old bytes need not be cleared: a new request gets its own page mapping and valid length, and must write every row before reading it.

Current decode calls `launchPagedAttention` to read the pages directly. `gatherKvCache` remains a helper for assembling a dense prefix; it is not part of the active decode attention path. Prefill still uses the dense score, mask, softmax, and value-mixing sequence.

How direct paged attention produces the same result

`pagedAttentionKernel` launches one block per active row and query head, with 64 threads representing the 64 head features. It maps that query head to its GQA KV head, walks only the sequence's valid cached positions, and computes scaled Q·K scores in FP32.

The kernel combines softmax and value mixing using running quantities rather than saving every score. For each score `s` and corresponding value feature `v`:

```
m_new = max(m, s)
correction = exp(m - m_new)
weight = exp(s - m_new)
denominator = denominator * correction + weight
numerator   = numerator   * correction + weight * v
m = m_new

final_feature = numerator / denominator
```

When a larger score arrives, `correction` rescales the previous partial sums to the new maximum. This gives the stable softmax-weighted sum without a `[1, L + 1]` probability buffer. Keys and values remain BF16, the running quantities are FP32, and the final attention output returns to BF16.

Paging changes where K/V live; this online softmax changes how attention is evaluated. Neither changes which earlier positions the new token is allowed to use. Numerical results can still differ from the older BF16 score-buffer path because the rounding points differ.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

9. How much memory does reuse cost?

Each processed token contributes K and V at every layer:

```
bytes_per_token = 2 × layers × KV_heads × head_dimension × bytes_per_value
                = 2 × 16 × 8 × 64 × 2
                = 32,768 bytes = 32 KiB
```

For 2,048 valid tokens, the K/V payload is `64 MiB`. With 32 KV heads instead of eight, the payload would be four times larger for the same layer count, head size, and dtype.

This figure excludes model weights, activation scratch, logits, attention scratch, and block-table metadata. The current dense prefill score allocation alone uses `32 × T_capacity² × 2` bytes: at capacity 2,048 that is `256 MiB`. Cache savings do not imply that every other allocation is small.

In the current pool, one physical page occupies:

```
2 × 16 × 512 × 2 = 32,768 bytes = 32 KiB
```

That is a page for one layer, whereas the earlier `32 KiB/token` figure spans all 16 layers. A 16-token span across all layers uses 16 pages, or `512 KiB`. A partly used final page consumes its full physical page space.

The pool defaults to `128 MiB` and is configurable with `--kv-cache-mib N`. Increasing request count does not automatically increase it. `KVCache::capacity == 2048` is the per-sequence logical limit, not a promise that every sequence owns 64 MiB immediately.

The scheduler separately reserves **admission credits** for the maximum prefix a request may process. Physical pages are assigned as that prefix grows. This keeps admitted requests from reaching a later page boundary with no room left to finish; the scheduling details continue in Part 9.

10. Count predictions and stop at the right time

The first prediction comes from prefill and counts toward `max_new_tokens`. A budget of one therefore requires no decode pass.

For prompt length `T` and an allowed output budget `B`, generating all `B` ordinary output tokens requires processing:

```
prompt tokens:             T
generated tokens fed back: B - 1
maximum cached prefix:    T + B - 1
returned output tokens:   B
```

The final predicted token need not be processed because we are not asking for another prediction. For `A, B, C` with budget two, prefill predicts `D`, decode processes `D` and predicts `E`, and generation returns `D, E`. The cache has processed four positions, not five.

Current `generateRequests` uses exactly `T + B - 1` when calculating each request's page budget. Video-era `main` allocated contiguous capacity `T + B`; it reserved one more token slot than the maximum processed prefix needs.

The current CLI clamps the allowed budget:

```
request.token_budget = std::min(
    max_new_tokens, MAX_SEQUENCE_LENGTH - prompt_length);
```

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

Here `prompt_length` includes the Instruct chat-format tokens produced by `encode_dialog_prompt`, not just the user-visible words. The fixed 2,048-token context contains the prompt and generated response together.

Current `accept_token`, a lambda inside `generateRequests`, handles predictions from prefill and decode consistently. It stops on the tokenizer's EOT or EOS IDs, or `END_OF_MESSAGE_TOKEN_ID` (`128008`, EOM). These stop IDs are not added to the visible generated-token vector or processed in another decode call. The video-era loop checked EOT and EOS; EOM is a later addition.

On ordinary output, the ID is appended and the request continues only while its output count is below its budget. On completion, its pages and admission credits return to the scheduler. Text is decoded from the entire output ID vector so byte pieces split across tokens can join into valid UTF-8.

11. Follow the current code by responsibility

Location / symbol	What to inspect
`src/main.cpp`: `main`	Parse the output/cache settings, chat-format prompts, clamp budgets, and allocate RoPE tables.
`KVCache`, `KVCachePool`, `KVBlockTable`	Separate sequence length/mapping from shared physical storage.
`KVPageAllocator::reserve`, `release`	Assign missing pages and return them on completion.
`prefill`	Reset a request cache, process prompt rows, append each layer's K/V, select the first output ID.
`decode`	Process one row per active sequence using each sequence's absolute position and block table.
`appendKvCache`	Translate logical positions into page offsets and copy K/V rows.
`generateRequests`, `accept_token`	Admit requests, feed predictions back, enforce stopping, and release pages.
`InferenceBuffers`, `freeInferenceBuffers`	Reuse scratch and one cuBLAS handle across layers and forward passes.
`src/kernels.cu`: `ropeKernel`, `launchRope`	Read explicit position IDs when rotating new Q/K.
`pagedAttentionKernel`, `launchPagedAttention`	Read per-sequence pages and compute an online softmax-weighted value sum.
`argmaxKernel`, `launchArgmax`	Pick one vocabulary ID per logit row; exact ties choose the smaller ID.

Most public launch declarations are in `include/kernels.cuh`. In this version, `launchPagedAttention` is instead forward-declared near the top of `src/main.cpp`; do not mistake its absence from the header for an unused kernel.

Current `projection_gpu` holds Q first, then the attention output projection, then the MLP down projection. `normalized_gpu` holds normalization output and later attention output. Reusing these buffers works because the earlier contents have already been consumed. These are temporary activations, separate from the K/V pages that must survive the step.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

12. Checks that make a cache bug visible

Run the project commands from the repository root on a supported NVIDIA CUDA machine with the matching model and tokenizer files installed as described in the README:

```
cmake -S . -B build
cmake --build build -j
./build/tokenizer-test models/llama-3.2-1b-instruct/tokenizer.model
./build/samir-vllm --stats "Explain the KV cache in two sentences." 1
./build/samir-vllm --stats "Explain the KV cache in two sentences." 32
```

The tokenizer executable checks tokenizer behavior; it does not verify CUDA attention or KV-cache numerics. A plausible generated response is a smoke check, not a numerical correctness test. No GPU validation result is claimed by these notes.

The one-output-token run exercises prefill and stopping without feedback decode; its `max_decode_batch` statistic should be zero. The larger budget allows decode unless the first prediction is a stop ID. After every completed workload, the reported `final_free_pages` should equal `pool_pages`, and the engine checks that all pages were returned.

For numerical validation, add a small temporary comparison harness around the existing forward functions. Their optional `logits_cpu` output copies FP32 logits back to the host; the normal CLI does not expose a logit-dump option.

Use a fixed list of token IDs and **teacher-force** the same next input ID into each path. Comparing independently generated responses first is harder to diagnose: one slightly different argmax changes the prefix for every later step.

Check	What it should expose
Prefill prefix P, decode a chosen token x, compare with fresh prefill of P + x	Cached attention should represent the same final position as recomputing the whole causal prefix.
Compare against an independent matching model on exactly the same IDs and positions	Catch errors shared by this engine's prefill and decode, such as a common RoPE mistake.
Inspect K/V after prefill and after one decode	Earlier rows should remain unchanged; the new row should appear at the old length in every layer.
Test cached lengths 15, 16, 17, 31, 32, 33	Expose page allocation, offset, and final-page-length errors.
Give a budget of one, then two; force a stop prediction in a harness	Check that prefill's prediction counts and the final/stop ID is not processed again.
Run different prompt lengths together, then reuse a completed slot	Expose position, row-to-cache mapping, or stale-context errors.

Compare intermediate tensors as well as final logits, starting with Q/K/V, RoPE, attention output, and each layer's residual output. Use tolerances informed by BF16 storage and the reduction path; record maximum/relative errors and near-tied top logits. Exact token agreement alone can hide meaningful tensor errors, and a near tie can change argmax despite small tensor differences.

The independent reference must match the checkpoint, chat formatting, RoPE settings, and input IDs. A raw prompt tokenization in one implementation and an Instruct dialog in the other are different inputs. Batched

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

BF16 GEMMs and the newer FP32 paged attention path can also round differently from their single-request or historical counterparts.

13. Keep these invariants beside the implementation

- Before processing a decode token: `0 < length < capacity`, and its absolute position is `length`.
- The new token's attention includes its own K/V, so it reads `length + 1` valid positions in each layer.
- Every layer writes to the same sequence position, using that layer's own weights and page mapping.
- Cached K is already rotated; cached V is unrotated; old cache entries are never rotated again.
- Scratch row order, position IDs, and block-table row order must describe the same active sequences.
- Valid length advances after a successful full forward pass, not after each layer or after merely predicting an ID.
- A completed request returns its pages before the slot is used for another prompt.

Caching removes repeated computation for earlier positions. It does not remove the new token's MLP or projections, and full-context attention still reads a growing prefix. That trade of GPU memory for reusable context is what makes repeated generation practical and what makes KV memory management central to serving several requests.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

Part 9: Static and Continuous Batching

Companion to Part 9: Static and continuous batching.

These notes explain how several requests share one model while keeping their histories separate. They also follow the repository beyond the original batching implementation into its current shared page pool and direct paged attention.

Read Part 8: Prefill, Decode, and the KV Cache first if the distinction between a processed token and a predicted token is new.

1. Which version are we explaining?

The batching implementation is recorded in commit `cfabb65` (`batching code`). That version has two sequence slots, `BATCH_SIZE = 2`, with a contiguous K/V allocation for each slot. It introduces packed decode rows, shared scratch buffers, static batching, continuous batching, and request completion handling.

The current implementation keeps those ideas but extends the memory system:

Original batching implementation	Current repository
Fixed two-slot scheduler	Runtime slot count derived from the workload and pool
Full contiguous cache allocated per slot	One fixed-size shared pool of physical KV pages
Per-request dense decode attention	Direct paged decode attention with online softmax
Slot availability controls admission	Slot availability and completion-budget credits control admission

Later implementation commits include `fa83cb7`, `fd823b9`, and `313999c`. The paging sections below describe current code additions, rather than claiming that every feature was already present at the Part 9 batching milestone.

The sources used here are the README, the batching and paging sections of the working notes, main.cpp, kernels.cu, and kernels.cuh. The explanation follows the checked-in implementation; it is not a GPU benchmark or a claim that reference-model correctness has been validated in this session.

2. Why batch decode at all?

After prefill, a single request usually contributes one token to each decode forward pass. Multiplying one hidden vector by a large weight matrix can leave GPU resources underused, while another request waits for its turn.

Batching places several independent token rows into the same operations. A shared weight matrix can serve all those rows, and the operation's launch overhead is spread over more useful work. The model weights are loaded once; we do not load another model for every user.

Keep three performance measurements separate:

- **Time to first token:** queue wait, prompt processing, and first prediction.
- **Time between tokens:** the gap between successive predictions for one user.
- **Throughput:** completed requests or generated tokens per unit of time.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

For an illustrative calculation, a 10 ms decode step producing one token gives 100 tokens/second. A 12 ms step producing two tokens gives about 167 tokens/second. Total token throughput improved, although each continuing request now waits 12 ms between those decode predictions. These are made-up numbers to explain the tradeoff, not measurements from this engine.

Batching does not make the dependency between one request's future tokens vanish. To predict token E after token D, that request still needs D's forward pass. The parallelism comes from doing useful work for other requests alongside it.

3. Batch size is different from sequence length

During prefill, `token_count = T` means T positions from one prompt. Dense causal attention connects those positions according to the prompt's causal mask.

During decode, `token_count = B` means one current token from each of B independent requests. Their prompt lengths and cache lengths can all differ.

Suppose request A has processed `[A, B, C]`, and request B has processed `[J, K, L, M, N]`. Their latest predictions are D and O:

```
Decode row 0: token D, request A, existing cache length 3
Decode row 1: token O, request B, existing cache length 5

token_ids    = [D, O]
position_ids = [3, 5]
```

After embedding gather, the shared activation matrix is `[2, 2048]`. The rows are two requests, rather than two adjacent positions in one conversation. We must never run a two-row causal attention calculation that lets O read D as though they belonged to the same sequence.

Shapes through one decode layer

The fixed model constants currently are:

Quantity	Value
Transformer layers	16
Hidden width	2048
Intermediate MLP width	8192
Query heads	32
KV heads	8
Head width	64
Vocabulary	128256
Context limit	2048 token positions

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

With B active requests, the principal tensor shapes are:

Operation or tensor	Shape
Hidden state and normalized hidden state	`[B, 2048]`
Q projection	`[B, 2048] = [B, 32, 64]`
New K and new V projections, each	`[B, 512] = [B, 8, 64]`
Attention output and output projection	`[B, 2048]`
Gate and up projections, each	`[B, 8192]`
MLP down projection	`[B, 2048]`
Final logits	`[B, 128256]`, stored as FP32
Greedy predictions	`[B]` token IDs

`projectBf16RowMajor` in `main.cpp` multiplies every input row by the same weights. Its cuBLAS transpose flags account for cuBLAS's column-major view of the row-major buffers; batching does not require transposing the input on GPU.

RMSNorm reduces across a row's hidden features. SwiGLU acts on corresponding elements of each row. Residual addition combines corresponding rows. None of these operations mixes the contents of different requests.

4. Follow one batched decode step

The entry point is `decode` in main.cpp. For our D/O example, one iteration performs the following work:

1. Validate each cache and record positions `[3, 5]` from its current length.
2. Ensure pages exist for prefixes of lengths `[4, 6]` in every layer.
3. Upload block tables in exactly the same packed order as `[D, O]`.
4. Gather both embeddings into the `[2, 2048]` activation buffer.
5. Run all 16 transformer layers in order.
6. Apply final RMSNorm, the LM head, and a separate argmax for each row.
7. Return the predictions and increment each cache length once.

Within each layer, Q/K/V projections run over both rows together. RoPE rotates D's Q/K at absolute position 3 and O's Q/K at absolute position 5. Those positions stay the same through all 16 layers; a layer is not another token position.

The new K/V rows are then copied to their owning sequence's pages. Cached keys already have RoPE applied. Old keys must not be rotated again, and V is not rotated. Attention includes the newly appended token:

```
D's query reads request A positions 0..3: A, B, C, D
O's query reads request B positions 0..5: J, K, L, M, N, O
```

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

Current decode launches paged attention for all active rows and heads. Each block follows one row's own table. Output projection and the MLP then process the resulting packed hidden rows together again.

If argmax returns `[E, P]`, the caches contain D and O, while E and P are merely predicted. The lengths become `[4, 6]`. If both requests continue, the next decode call consumes `[E, P]` at positions `[4, 6]`.

5. Request identity, slot, and packed row

These are different kinds of identity, even when their numbers happen to match:

Name	What it identifies	Current representation
Request index	One input job and its output	Index into `requests`
Sequence slot	Active ownership and cache metadata	Index into `caches` and `slot_requests`
Packed batch row	One row in this decode call	Index into `active_tokens` and `active_caches`

Initially, A might occupy slot 0 and B slot 1. If A finishes, only B remains:

```
slot_requests = [-1, B]
active_slots  = [1]
active_tokens = [B's latest prediction]
active_caches = [&caches[1]]

Packed row 0 -> slot 1 -> request B
```

When decode returns row 0's prediction, `active_slots[0]` finds slot 1, and `slot_requests[1]` finds request B. Treating output row 0 as slot 0 would send the token to the wrong owner.

If C later occupies slot 0, the next packed order can be `[C, B]`. B's logical history has not moved; its temporary row number changed because the batch did.

The same order must hold for token IDs, positions, cache pointers, GPU block tables, logits, and returned token IDs. Current `decode` also rejects duplicate cache pointers within one batch and requires every cache to use the same pool.

6. Buffer reuse is about lifetime

`InferenceBuffers` owns reusable GPU scratch and one cuBLAS handle. Prefill allocates scratch when first needed. Later prompts, layers, and decode steps reuse that storage. Reusing storage means calculating fresh results into it; it does not mean reusing another request's hidden representations.

Current capacity fields have different purposes:

- `prefill_capacity`: the largest encoded prompt length in the workload.
- `batch_capacity`: the runtime maximum number of simultaneously fitting requests.
- `token_capacity`: the larger of those two capacities, for shared row buffers.

The scheduler sizes the possible batch by sorting required page budgets and counting the largest group of smallest budgets that fits. This is only a scratch sizing calculation. The actual request queue remains FIFO.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

When a later workload needs larger capacities, `generateRequests` frees the old buffer set before updating its capacities. Changing the recorded capacity without reallocating would let later kernels write beyond an old allocation.

Safe aliases inside a transformer layer

Storage	Earlier contents	Safe later use
`normalized_gpu`	Input RMSNorm output	Attention output, after Q/K/V have read the norm
`projection_gpu`	Q	Attention output projection, after attention has read Q
`projection_gpu`	Attention output projection	MLP down projection, after the first residual reads it
`gate_gpu`	Gate projection	SwiGLU result, after consuming its original gate values

The residual's base hidden state must survive until its update has been added. Likewise, the normalized input must survive both gate and up projections. The rule is to overwrite a value only after its final reader has finished.

The current operations use ordered CUDA/cuBLAS execution and explicit checks; decode synchronizes after each layer. Adding independent streams later would require explicit dependencies before assuming these aliases remain safe.

The KV pool has a different lifetime. It preserves history across decode steps, while scratch can be overwritten during the next operation. Returning a finished request's pages makes them available to another request, but does not erase the bytes. New owners write valid positions before attention is allowed to read them.

Only dense prefill allocates the `[32, T_capacity, T_capacity]` BF16 attention-score scratch. Current paged decode does not use that score matrix, nor does its K/V scratch need room to gather an entire cached history.

7. Completion and the first-token accounting trap

`prefill` already predicts the first output token. `accept_token`, the completion handler inside `generateRequests`, handles predictions from both prefill and decode. This gives them the same stopping and budget rules.

Current stop checks recognize tokenizer EOT, tokenizer EOS, and `<|eom_id|>` (`128008`). A stop token completes the request and is not appended to the visible generated token list. Other predictions are appended before the budget check.

For a three-token prompt and an output budget of three:

Work completed	Prediction	Visible generated tokens	Processed cache length
Prefill A, B, C	D	D	3
Decode D	E	D, E	4
Decode E	F	D, E, F	5

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

The request completes after F is predicted. Processing F would only be useful if we wanted to predict another token after it.

Let P be the encoded prompt length and G the allowed output budget. The largest processed prefix before completion is therefore:

```
maximum_cached_tokens = P + G - 1
```

For `G = 1`, prefill can complete the request without any decode call. For an immediate stop prediction, its response can contain zero visible tokens.

`main` first encodes an Instruct chat prompt, including the headers and special tokens. Its P is not the word count or raw text token count. It requires `P < 2048` and computes:

```
G = min(requested_max_new_tokens, 2048 - P)
```

This is the current public context policy. Do not silently change the formula to allow one more returned token just because the final prediction is uncached.

On completion, the scheduler prints the whole decoded response, releases all assigned pages through `freeKvCache`, returns the admission credits, and marks the slot `-1`. Decoding the entire generated byte sequence also lets split UTF-8 token fragments join correctly. The CLI currently prints finished responses; it does not stream each token to a client as it is produced.

8. Static and continuous scheduling

Both policies use the same batched decode. Their difference is when waiting requests are admitted, as implemented in `generateRequests`.

Assume A and B are already prefilled and admitted. A needs two further decode steps, B needs six, and C is waiting. This illustrative schedule assumes enough page credits for C once A completes.

Static batching

The scheduler admits a group, then waits until every member finishes before admitting another group:

```
Decode round      1  2  3  4  5  6
Slot 0            A  A  -  -  -  -
Slot 1            B  B  B  B  B  B
C waiting         C  C  C  C  C  C

After round 6: prefill C, then start the next group.
```

Finished requests leave subsequent packed decode calls. Static batching does not keep running fake rows for them; it simply leaves the freed slot unused until the group drains. That shrinking batch can reduce useful GPU work.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

Continuous batching

The scheduler checks admissions between decode iterations, so C can enter after A completes while B keeps its own cache:

```
Decode A+B -> Decode A+B -> A completes
                              |
                           Prefill C   (B waits)
                              |
Decode C+B -> Decode C+B -> Decode C+B -> ...
```

The decode batch membership changes over time. Continuous admission can make better use of available capacity when responses have different lengths.

There is still one serial prefill per newly admitted prompt. A long prefill can delay B's next token, and several admissions can run several prefills before the next decode iteration. The code does not overlap prefill with decode, split prefills into chunks, or run an asynchronous request-serving API.

One scheduler iteration

```
If continuous mode OR every current slot is empty:
    Admit FIFO requests while there are free slots and enough page credits.
    Prefill each admitted prompt; immediately handle its first prediction.

Pack only active slots into token IDs, cache pointers, and row mappings.
If nobody is active and the queue is empty: finish.
Otherwise run one batched decode step.
Map output rows back to requests and handle completions.
Repeat.
```

In continuous mode a request that completes during prefill can let another queued request use that slot immediately. Static mode breaks after one admission attempt per slot; its next group still waits for the active group.

9. The shared pool: memory follows processed history

At the original two-slot milestone, a full contiguous cache costs:

```
Per token, one layer: 2 (K and V) * 512 values * 2 BF16 bytes = 2048 bytes
Per token, 16 layers: 2048 * 16 = 32768 bytes = 32 KiB
Full 2048-token sequence: 2048 * 32 KiB = 64 MiB
Two full sequence allocations: 128 MiB
```

Short requests used only a small portion of those full allocations. Current `KVCachePool` instead allocates one shared 128 MiB pool by default. Adding more input requests does not multiply this pool allocation.

A physical page belongs to one sequence and one layer while assigned. Its layout is K for 16 positions followed by V for the same 16 positions:

```
K half: [16, 512] BF16 = 16384 bytes
V half: [16, 512] BF16 = 16384 bytes
Physical page total  = 32768 bytes = 32 KiB

128 MiB / 32 KiB = 4096 physical pages in the default pool
2048 / 16 = 128 logical pages per sequence, per layer
16 layers * 128 pages = 2048 pages for one full-context cache
```

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

One 16-token logical span across all 16 layers needs 16 physical pages, or 512 KiB. Do not confuse a 32 KiB physical page with a complete 16-layer span.

`KVBlockTable` holds `[16, 128]` page IDs for each sequence. Unassigned entries are `-1`. For layer 0, a possible mapping is:

```
Logical page 0 -> physical page 5   (positions 0..15)
Logical page 1 -> physical page 1   (positions 16..31)
Logical page 2 -> physical page 7   (positions 32..47)
```

Position 33 is in logical page `33 / 16 = 2`, at offset `33 % 16 = 1`. The table routes it to physical page 7, at that same token offset. Layer 1 has its own mapping and its own values.

This is an application-managed mapping inspired by virtual memory. It does not make GPU memory automatically spill to CPU RAM, and it is not OS page faulting. The pool itself is one contiguous GPU allocation; a sequence's logical history is distributed among pages within it.

10. Admission credits prevent future pool exhaustion

Lazy assignment alone is insufficient. If the scheduler admits requests based only on pages used right now, several growing requests could later need more pages than remain available.

The current scheduler commits enough credits for each admitted request to finish its declared output budget:

```
cached_budget = P + G - 1
pages_per_layer = ceil(cached_budget / 16)
required_pages = pages_per_layer * 16

Admission condition:
required_pages <= pool_page_count - committed_pages
```

For `P = 40` and `G = 25`, the largest processed prefix is 64 tokens. It needs four pages per layer, or 64 physical pages = 2 MiB of credits. Its prefill initially assigns only three pages per layer, or 48 pages = 1.5 MiB. At processed position 48, it receives another page per layer using credits that were already committed. Admission credits and assigned pages are different.

`KVPageAllocator::reserve` counts all missing pages across all layers before changing any table entries. If too few physical pages exist, it returns failure without partially updating that table. Prefill reserves enough for the prompt; decode reserves enough for each current `length + 1`.

When a request ends early, both its assigned pages and its full credit commitment are released. Unused credits were never physical page assignments.

FIFO admission does not skip a large head request to admit smaller requests behind it. This can leave capacity temporarily unused. A request whose complete budget exceeds the whole pool is rejected before generation starts, with a message suggesting a smaller output budget or larger pool.

Within this policy, active requests have room to finish. The scheduler needs no preemption, CPU swapping, or recomputation to resolve later page-boundary growth. The cost is conservative concurrency: a large requested output allowance consumes credits even if the model will probably emit a stop token much earlier.

11. Direct paged attention and online softmax

Paging answers where K/V live. Attention still needs one normalized weighted sum over the sequence's entire valid history. Current decode follows the pages directly instead of gathering the prefix into a contiguous temporary cache.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

The implementation is `pagedAttentionKernel` and its `launchPagedAttention` wrapper in kernels.cu. Unlike the other public launchers, this wrapper is currently forward-declared near the top of `main.cpp`, rather than declared in `kernels.cuh`.

Threads, heads, and addresses

The launch uses a grid `[B, 32]` and 64 threads per block:

- `blockIdx.x` selects the packed request row.
- `blockIdx.y` selects one of 32 query heads.
- `threadIdx.x` selects one of the head's 64 components.

Each block handles one query head. GQA maps it to KV head `query_head / 4`: query heads 8, 9, 10, and 11 all use KV head 2. They still have distinct queries and attention outputs.

The valid history length is `position_ids[row] + 1`, including the just-appended token. The last page is read only up to that valid count. No future-token mask is necessary because decode has one query and reads only its existing prefix.

For physical page p, token offset t, KV head h, and component d, element offsets within the BF16 pool are:

```
page_start = p * (2 * 16 * 512)
key_index  = page_start + t * 512 + h * 64 + d
value_index = key_index + (16 * 512)
```

For every cached position, each thread multiplies its Q component by the matching K component. Warp shuffles reduce the two 32-thread halves; shared memory combines them into the 64-component dot product. Dividing by 8 applies the attention scale `1 / sqrt(64)`.

Keep a running normalized sum

For scores `s_i = dot(q, k_i) / 8`, ordinary stable attention computes:

```
m = max(s_i)
output = sum(exp(s_i - m) * v_i) / sum(exp(s_i - m))
```

Online softmax maintains the same information while reading positions one at a time. Let m be the running maximum, l the scaled denominator, and a the scaled weighted-value sum. Initialize `m = -infinity`, `l = 0`, `a = 0`. For a new score s and value vector v:

```
m_new = max(m, s)
correction = exp(m - m_new)
weight = exp(s - m_new)

l = l * correction + weight
a = a * correction + weight * v
m = m_new

After processing all valid positions: output = a / l
```

The weighted sum a has 64 components. Each thread owns one component in an FP32 register; the maximum and denominator are also maintained in FP32. If a larger maximum arrives, both old sums are rescaled onto the new common exponential scale. Rescaling only the denominator would change the result.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

For a scalar illustration, use scores `[1, 2, 1]` and values `[10, 20, 40]`:

Position	Running maximum	Correction	New weight	Denominator	Weighted sum
First	1	0	1	1	10
Second	2	0.367879	1	1.367879	23.678794
Third	2	1	0.367879	1.735759	38.393972

The result is approximately `38.393972 / 1.735759 = 22.1194`. It matches normalizing the three weights `[exp(-1), 1, exp(-1)]` together. Real attention runs the value accumulation independently for all 64 components.

Crossing a page boundary changes the K/V address, not the normalization problem. The kernel retains m, l, and a while following the next logical page's mapping. Normalizing each page separately and averaging page outputs would generally be wrong because pages can contribute different total softmax probability mass.

This implementation avoids a decode score buffer and contiguous KV gather. It still visits every valid cached position for each query head. Its two-warp block and serial loop over positions are a teaching implementation, rather than a claim of matching highly tuned production attention kernels.

12. Running the scheduler and reading its statistics

With the dependencies and matching model files described in the README:

```
cmake -S . -B build
cmake --build build -j

# Continuous batching is the default; each request gets up to 64 new tokens.
./build/samir-vllm --continuous --stats "Say hello." 64 "Explain GQA." "What is RoPE?"

# Admit a new group only after the previous group has drained.
./build/samir-vllm --static --stats "Say hello." 64 "Explain GQA." "What is RoPE?"

# Keep the same workload but reduce the shared pool to 8 MiB.
./build/samir-vllm --kv-cache-mib 8 --stats "Say hello." 64 "Explain GQA." "What is RoPE?"
```

Flags precede positional arguments. The first positional argument is a prompt, the second is the output budget, and any remaining arguments are more prompts. That one requested budget applies to every request before context clamping. Use `--` before a prompt beginning with `--` so it is not parsed as an option.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

`--stats` prints a JSON object after successful generation:

Field	Meaning
`pool_bytes`	Bytes allocated for the shared KV data pool
`pool_pages`	Number of physical 32 KiB pages in that pool
`peak_active`	Highest simultaneous admitted request count, including prefill
`max_decode_batch`	Largest actual number of rows passed to decode
`peak_pages`	Highest number of physically assigned pages observed
`admission_waits`	Failed FIFO-head admission attempts because credits did not fit
`final_free_pages`	Physical pages returned to the allocator at completion

`peak_active` can exceed `max_decode_batch`: an admitted request can finish on its prefill prediction and never enter a decode batch. `admission_waits` is a count of attempts, not elapsed waiting time or a count of distinct blocked jobs.

The final free count should equal the pool page count. The engine checks that all physical pages and committed credits were returned before reporting success. The pool figure excludes model weights, scratch, tables, and other GPU memory.

13. Correctness and performance checks worth making

Reasonable-looking text is useful feedback, but it cannot prove that packed rows use the right cache. On a supported target GPU, compare intermediate tensors and logits with an independent implementation using the exact same checkpoint and encoded token IDs.

Exercise different prompt lengths, a request finishing while another continues, replacement of a low-numbered slot, and appends across positions 15/16 and 31/32. Also cover a one-token output budget, immediate stop, early page release, FIFO admission waits, and rejection of a budget larger than the entire pool.

For a paged-attention reference, gather only the valid K/V prefix and compare its dense FP32-softmax result with the direct paged result within a justified tolerance. Match GQA head mapping, RoPE positions, scale, and inclusion of the current token. `gatherKvCache` remains in `main.cpp` as a dense reference helper; current decode does not call it.

BF16 projections can round differently at different batch sizes. When two vocabulary logits nearly tie, a small difference can change greedy argmax, and the resulting generations can diverge. The argmax kernel's smallest-ID tie rule only resolves exactly equal logits; it cannot make near ties batch invariant. Inspect numerical error and the top-logit gap when comparing runs.

Measure queue delay, prompt time, inter-token gaps, and total throughput separately. A larger decode batch can improve total tokens/second while serial prefill increases one continuing user's next-token delay. These notes provide the code model and suggested checks, not measured performance results.

14. Reading the code with a purpose

Follow this route through main.cpp:

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

1. Read `Request`, `KVCache`, `KVBlockTable`, and `InferenceBuffers` to identify ownership.
2. Read `main` to see chat formatting and output-budget clamping.
3. Read the start of `generateRequests` for credit accounting and runtime sizing.
4. Follow `accept_token` for completion and page release.
5. Follow the admission loop, then `active_slots`, then the returned-row mapping.
6. Read `decode` to connect packed rows to positions and uploaded tables.
7. Read `appendKvCache` and `KVPageAllocator::reserve` for physical storage changes.
8. Read `pagedAttentionKernel` in kernels.cu for direct page traversal.

The central invariant is that batching shares model computation while every row retains its own request identity, absolute positions, and valid history. The scheduler decides which requests may run; the page allocator supplies their storage; attention follows each row's mappings to read that request's context.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

CUDA Kernels - Getting Started Notes

Why CUDA is part of this project

An LLM inference engine spends most of its time doing large amounts of numerical work on model weights and intermediate values. A CPU is very good at running a small number of complicated tasks quickly. A GPU is built to run a huge number of simpler tasks at once.

That does not mean a GPU magically makes everything faster. The work has to be split into lots of pieces that can run independently. CUDA is NVIDIA's programming model and toolchain for writing that GPU work from C++.

In this engine, C++ does the host-side work:

- load and parse model files
- tokenize prompts
- allocate and manage memory
- decide what operation runs next
- launch GPU work

CUDA kernels do the device-side work:

- gather embedding values
- normalize activation vectors
- run matrix operations
- apply attention
- sample or select the next token

The basic setup is not "replace C++ with CUDA." It is one program with two sides:

```
CPU / host: organize work and launch operations
GPU / device: execute lots of numerical work in parallel
```

The first thing to keep straight: host vs device

Host memory is normal CPU RAM. Device memory is GPU VRAM. They are separate memory spaces.

```
CPU code can directly access host memory.
GPU kernels can directly access device memory.
```

So this is not valid as a general assumption:

```
"I have a C++ vector, so the GPU kernel can read it."
```

It cannot. First allocate GPU memory, then copy the data over.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

```
int *token_ids_gpu = nullptr;

cudaMalloc(&token_ids_gpu, count * sizeof(int));

cudaMemcpy(
    token_ids_gpu,
    token_ids_cpu,
    count * sizeof(int),
    cudaMemcpyHostToDevice);
```

The same idea applies to outputs. A kernel can write output into GPU memory. We only copy that output back to CPU memory if the CPU actually needs it. In inference engines, keeping data on the GPU between operations is important. Copying activations back and forth after every layer would destroy performance.

Think of CPU RAM and GPU VRAM as two workshops with a transport lane between them. Moving a small list of token IDs is fine. Moving a whole room of model weights repeatedly is not. Bring the model into the GPU workshop once, then keep using it there.

What a kernel actually is

A CUDA kernel is a C++-like function that executes on the GPU.

```
__global__ void addKernel(const float *a, const float *b, float *out, int count)
{
    const int index = blockIdx.x * blockDim.x + threadIdx.x;

    if (index >= count)
    {
        return;
    }

    out[index] = a[index] + b[index];
}
```

`__global__` says this function is launched by the host and runs on the device.

The code looks like it runs once, but when we launch it, the GPU runs many copies of it. Every copy is a separate GPU thread with its own `threadIdx` and therefore its own `index`.

If `count` is one million and we launch at least one million threads, then thread 0 adds item 0, thread 1 adds item 1, and so on.

This way of thinking is more useful than thinking "a kernel runs on every GPU core." You write a thread program. CUDA and the GPU scheduler take care of running a huge number of those threads across the actual hardware.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

CUDA's hierarchy: thread, block, grid

CUDA organizes a launch into three levels:

```
grid
  block 0
    thread 0
    thread 1
    ...
  block 1
    thread 0
    thread 1
    ...
```

- A **thread** is one execution of the kernel.
- A **block** is a group of threads that can cooperate with each other.
- A **grid** is every block in one kernel launch.

For a one-dimensional array, this is the standard global index calculation:

```
const int index = blockIdx.x * blockDim.x + threadIdx.x;
```

Example launch:

```
3 blocks
4 threads per block
```

Block	Thread	Global index
0	0	0
0	1	1
0	2	2
0	3	3
1	0	4
1	1	5
1	2	6
1	3	7
2	0	8
2	1	9
2	2	10
2	3	11

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

`blockIdx.x` tells a thread which group it belongs to. `threadIdx.x` tells it where it is inside that group. `blockDim.x` tells it how many threads are in each block.

CUDA also supports `x`, `y`, and `z` dimensions. We can use a 2D or 3D layout when the data naturally fits that shape. But a 1D flattened index is often simpler and is exactly what the embedding gather kernel uses.

Launching a kernel

Host code launches a kernel with CUDA's triple-angle-bracket syntax:

```
addKernel<<<blocks, threads_per_block>>>(a_gpu, b_gpu, out_gpu, count);
```

The first value is how many blocks to launch. The second is how many threads are in each block.

For a one-thread-per-element kernel, calculate the number of blocks with ceiling division:

```
const int threads_per_block = 256;
const int blocks =
    (count + threads_per_block - 1) / threads_per_block;

addKernel<<<blocks, threads_per_block>>>(a_gpu, b_gpu, out_gpu, count);
```

Why round up? We have to launch enough threads for the final element even if the length is not a multiple of 256.

```
count = 1,000
threads per block = 256
blocks = (1000 + 255) / 256 = 4
threads launched = 4 * 256 = 1,024
```

The final 24 threads have no useful element to process. This is normal. It means the kernel needs a bounds check.

Bounds checks are not optional

Every simple elementwise CUDA kernel should usually have this near the top:

```
if (index >= count)
{
    return;
}
```

This lets unused threads in the final block exit safely.

Without it, a thread could write to `out[1003]` when an output array only has valid indices `0` through `999`. GPU out-of-bounds access can give wrong output, an illegal-memory-access error, or something that looks fine until a later unrelated kernel fails. That makes it annoying to debug.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

The common kernel pattern

Most first CUDA kernels follow this structure:

```
__global__ void kernelName(
    const InputType *input,
    OutputType *output,
    int count)
{
    const int index = blockIdx.x * blockDim.x + threadIdx.x;

    if (index >= count)
    {
        return;
    }

    output[index] = someOperation(input[index]);
}
```

Then a host launcher calculates the grid size and launches it:

```
cudaError_t launchKernel(
    const InputType *input,
    OutputType *output,
    int count)
{
    const int threads_per_block = 256;
    const int blocks =
        (count + threads_per_block - 1) / threads_per_block;

    kernelName<<<blocks, threads_per_block>>>(input, output, count);
    return cudaGetLastError();
}
```

This is not the only way to write CUDA, but it is a good mental starting point. First make the work mapping obvious. Then verify it is correct. Then think about performance.

Why keep a launcher separate from the kernel

We could write `kernel<<<...>>>(...)` directly in `main.cpp`, but a launcher is cleaner for an inference engine.

- The rest of the program calls a normal C++ function such as `launchEmbeddingGather`.
- The launch geometry lives next to the kernel implementation.
- We get one place to add launch-error handling.
- We can change the block size, add a CUDA stream, or tune the launch later without changing every caller.

The launcher is the boundary between CPU orchestration and GPU computation.

```
host code
-> launcher decides how many GPU threads are needed
-> kernel runs those threads
-> GPU output stays in VRAM for the next kernel
```

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

Threads do not always mean one element

One thread per output element is a good first mapping, but it is not a rule.

For embedding gather, one thread writes one BF16 output value. That works because every value is independent.

For RMSNorm, one output vector needs a sum across all 2,048 values. Threads need to collaborate, so the mapping changes: one block handles one token and every thread works on two values. The threads then combine partial sums through a reduction.

For matrix multiplication, a thread may calculate one output value, several values, or part of a tile. Libraries like cuBLAS have heavily tuned implementations for this, which is why we will use them for the big matrix multiplications rather than assume a basic custom kernel will beat them.

The question before writing a kernel is always:

```
What is the smallest independent piece of output work,
and what data does that thread need to produce it?
```

Global memory, shared memory, and registers

There are different places a GPU thread can keep data.

Global memory

Global memory is the big VRAM pool. Kernel arguments like `embed_tokens`, `token_id_gpu`, and `activations_gpu` point into global memory.

- Large capacity.
- Visible to all threads and kernels.
- Much slower than on-chip storage.

Model weights and large activation tensors live here because they are too large for anything else.

Registers

Registers are tiny, very fast storage private to one thread.

```
const int index = ...;
const float x = input[index];
```

Values like `index` and `x` will commonly live in registers when possible. Other threads cannot directly read a thread's registers.

Shared memory

Shared memory is small, fast storage shared by threads in the same block.

```
__shared__ float partial[1024];
```

It is useful when threads in a block need to cooperate, like calculating a sum, max, or reduction. RMSNorm uses it for partial squared sums.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

Shared memory is not shared across the whole grid. A thread in block 0 cannot use shared memory from block 1. This block-level boundary is one of the main things to understand before writing reductions and attention kernels.

Synchronization

Threads in a block do not run one after another in a predictable order. If one thread writes shared memory and another thread immediately reads it, we need a barrier to make sure all relevant writes are done first.

```
__syncthreads();
```

This means: every thread in this block must reach this point before any can continue past it.

Think of it like a group project checkpoint. Nobody starts using the shared whiteboard results until everyone has written their assigned part.

`__syncthreads()` only works for threads in the same block. There is no simple normal barrier for every thread in an entire grid during one standard kernel launch. If blocks must communicate, we normally use multiple kernel launches or a different algorithm.

One important rule: do not put `__syncthreads()` inside a branch that only some threads take. Every thread in the block must reach it, otherwise the block can hang.

Warps: the group the GPU actually schedules together

CUDA gives us threads and blocks, but the GPU hardware typically executes threads in groups of 32 called warps.

```
one block of 256 threads
= 8 warps of 32 threads
```

The threads in a warp normally execute the same instruction together. If an `if` statement sends some warp threads one way and other threads another way, the GPU usually has to execute both paths while masking off the threads that are not active for that path. This is called warp divergence.

Example:

```
if (index % 2 == 0)
{
    doA();
}
else
{
    doB();
}
```

This is not automatically wrong. It is just work to be aware of when optimizing. The bounds check in a final partial warp or final partial block is normal and usually worth the simplicity.

For now, the useful rule is: nearby thread indices often run together, so give nearby threads similar work and nearby memory accesses when you can.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

Memory access patterns and coalescing

GPUs are fast when neighboring threads access neighboring addresses in global memory. Hardware can combine those reads or writes into fewer memory transactions. This is called coalescing.

Good pattern:

```
out[index] = input[index];
```

If threads 0 through 31 run together, they access 32 adjacent values.

Less friendly pattern:

```
out[index] = input[index * 2048];
```

Now neighboring threads jump far apart through memory.

Embedding gather is partly irregular because different prompt positions may select totally different vocabulary rows. But inside one selected row, it is good: adjacent threads load adjacent hidden values. In this engine, 256 threads per block divides the 2,048 hidden size exactly, so a block handles a contiguous section of one embedding row.

This is a useful distinction: we cannot always make every access perfect, but we can make the contiguous part of the work simple and GPU-friendly.

CUDA is asynchronous

When the CPU calls:

```
embeddingGatherKernel<<<blocks, threads_per_block>>>(...);
```

the call usually queues GPU work and returns before the GPU has finished. This lets the CPU keep organizing later work instead of waiting around.

There are two different error moments to understand:

```
return cudaGetLastError();
```

This checks whether launching the kernel failed immediately, for example because the launch configuration was invalid.

```
cudaDeviceSynchronize();
```

This waits for earlier GPU work to complete. It can surface errors that happened while the GPU was actually executing the kernel, such as an illegal memory access.

During early development, synchronization makes debugging much easier because an error is reported close to the kernel that caused it. In a more optimized engine, we avoid unnecessary synchronization because making the CPU wait kills overlap and throughput.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

A minimum debugging checklist

When a first kernel gives wrong results, check these before trying to do clever optimization:

1. Is every input pointer device memory, not a CPU pointer?
2. Did the `cudaMalloc` byte count use `element_count * sizeof(type)`?
3. Is the host-to-device or device-to-host direction correct in `cudaMemcpy`?
4. Does every thread calculate a unique valid index?
5. Is there a bounds check for rounded-up launches?
6. Is the logical matrix shape consistent with the flattened index calculation?
7. Do input and output use the intended dtype, for example `int` IDs and BF16 activations?
8. Did we call `cudaGetLastError()` after the launch?
9. When debugging, did we synchronize before trusting the result?
10. Can we compare a tiny test case against a CPU or PyTorch reference result?

Small tests are way easier to reason about. Before a 128,256 by 2,048 table, make a fake table with four rows and four columns. Then manually verify every output value.

Things to avoid early on

- Do not optimize before checking correctness against a reference.
- Do not use CPU pointers inside a kernel.
- Do not forget that CUDA launches are asynchronous.
- Do not assume every model has a 2,048 hidden size. This project hard-codes it because it targets one Llama model.
- Do not assume all GPU threads can synchronize with all other threads. Synchronization and shared memory are block-local.
- Do not use BF16 blindly for every intermediate calculation. Numerical operations such as sums and reductions often need FP32 accumulation even when the input and output are BF16.
- Do not write a custom matrix multiplication just because it is possible. Start with cuBLAS for the big GEMMs and only write custom code when there is a specific reason.

How this grows into an inference engine

The embedding gather is the easy first kernel: independent reads and writes.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

The next operations introduce the more interesting GPU concepts:

```
Embedding gather
  one thread per output value
  no thread cooperation

RMSNorm
  one block per token
  shared memory and reduction

Q, K, V and MLP projections
  large matrix multiplications
  cuBLAS GEMMs

RoPE, residual adds, SiLU, softmax
  custom elementwise or reduction-style kernels

Attention
  careful memory layout, reduction, causal masking, and eventually KV cache access
```

The CUDA basics do not change. Every operation still needs us to decide:

1. What data is on the GPU?
2. What output are we producing?
3. How do we split that output into thread-sized pieces?
4. Do threads need to communicate?
5. What memory pattern will those threads create?
6. How do we validate the result before optimizing it?

That is really the low-level job of an inference engine. The model gives us the weights and the math. CUDA lets us decide how that math is mapped onto the GPU.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.